



Inverse Optimal Transport in Cosmological Simulations

THESIS

submitted in partial fulfillment of the
requirements for the degree of

BACHELOR OF SCIENCE

in

MATHEMATICS AND ASTRONOMY

Author :

B.A. Sturre

Student ID :

3635066

Supervisor Astronomy:

Dr. M. Schaller

Supervisor Mathematics:

Dr. E. Sellentin and
Dr. D. van der Hoeven

Leiden, The Netherlands, June 13, 2025

Inverse Optimal Transport in Cosmological Simulations

B.A. Sturre

Leiden Observatory, Leiden University
Mathematical Institute, Leiden University
P.O. Box 9500, 2300 RA Leiden, The Netherlands

June 13, 2025

Abstract

The theory of Optimal Transport (OT) provides a powerful mathematical framework for understanding dynamics, with applications in various disciplines. In cosmology, OT can offer a unique view through which to analyze structure formation and evolution, by connecting initial and final density configurations. In this thesis, we investigate the structure formation in cosmology using an inverse OT approach, focusing on the movement of particles. We utilize data from the FLAMINGO simulations, segmented into smaller boxes for computational efficiency. Our primary objective was to infer the cost matrix that is responsible for the particle motion within these simulations. We developed the theory of inverse OT and developed an algorithm to solve for the cost matrix, while addressing the ill-posed nature of inverse OT. This algorithm is applied to both reference cases and the FLAMINGO simulation data. The derived cost matrix shows a high degree of symmetry, consistent with an isotropic universe. As a proof of concept, a Markov Chain Monte Carlo simulation was done to test if the FLAMINGO cost matrix is a linear combination of reference cases. These Markov chains did not converge. However, this is thought to be because of the limited number of reference cases.

Contents

1	Introduction	1
2	Theoretical Foundations of Optimal Transport	3
2.1	Measure Theory	3
2.2	Monge-Kantorovich Minimization Problem	4
2.3	Discrete Optimal Transport	5
2.3.1	Solving the Monge-Kantorovich Minimization Problem	6
2.4	Entropy Regularization	7
2.4.1	Sinkhorn Scaling	8
2.5	Inverse Optimal Transport	10
3	Data	13
3.1	FLAMINGO	13
3.1.1	Boxes	14
4	Algorithm for Inverse Optimal Transport	15
4.1	The Dual Formulation of Inverse Optimal Transport	16
4.1.1	Equivalence of Solutions	17
4.2	Algorithm Development	18
4.3	Algorithm Implementation	19
4.3.1	Numerical Stability	19
4.3.2	Minimization and choosing $R(c)$	20
5	Recovering the Cost Matrix	23
5.1	Reference Cases	23
5.1.1	Linear Motion	23
5.1.2	Voids	25
5.1.3	Clusters	27
5.1.4	Walls	29
5.1.5	Filaments	31
5.2	FLAMINGO Cost Matrix	31
5.2.1	Expected Cost Matrix	31
5.2.2	Recovered Cost Matrix	32
5.2.3	Markov Chain Monte Carlo Simulation	32
6	Outlook	37
6.1	Future Directions	37
6.2	Acknowledgments	38
	References	39

A	Proofs	43
A.1	Proof of Lemma 4.1	43
A.2	Proof of Theorem 4.2	45
A.3	Proof of Theorem 4.4	46
A.4	Proof of Lemma 4.5	48
B	Differences Between Algorithms	55
C	Additional Reference Cases	57
C.1	Voids	57
C.2	Clusters	58
C.3	Walls and Filaments	59

1. *Introduction*

In the early universe, there were small irregularities in the density of matter. Today, these irregularities can be seen in the Cosmic Microwave Background Radiation (Durrer, 2015). When gravity starts to act on these small fluctuations, regions that have a slightly higher density will exert a stronger gravitational pull, attracting more matter. This gravitational pull starts off a process where the dense regions will grow denser and the less dense regions will get emptier. Over time, this process leads to the formation of over-dense regions, clusters, and under-dense regions, voids (Zel'dovich, 1970). The formation of these structures have been studied extensively with the use of hydrodynamical simulations (Schaye et al., 2014; Crain et al., 2015; Schaye et al., 2023; Kugel et al., 2023), as it is difficult to observe this structure formation directly (Bos, 2016).

These hydrodynamical simulations have an initial and final particle distribution, often with in-between time-steps. This provides a natural framework for Optimal Transport theory (OT), a rapidly developing area of mathematics. OT allows us to compute distances and relationships between probability distributions (Villani, 2003). In OT, one looks for the transport of some type of particles (from the initial to the final positions), and tries to minimize the cost this transport takes, making use of some cost function. This cost function is often taken as the work it takes to move particles, possibly with other transport costs, such as fuel costs (Villani, 2008).

In simulations governed by physical laws, the exact form of cost function is typically unknown, due to the complex gravitational dynamics and non-linear evolution of structures. The reconstruction of initial conditions has been studied within the field of cosmology (Nikakhtar et al., 2024), but the inverse OT problem has to our knowledge not been applied in the setting of structure formation in cosmology. Therefore, the goal of this thesis is to reverse the optimal transport process; can we infer the cost function from the observed particle movements?

To reverse the OT process, data of the FLAMINGO (Full-hydro Large-scale structure simulations with All-sky Mapping for the Interpretation of Next Generation Observations) simulations will be used, see also Schaye et al. (2023); Kugel et al. (2023). There are different algorithms and methods to recover the cost matrix, each designed for different constraints (Stuart and Wolfram, 2020; Ma et al., 2020; Li et al., 2019; Chiu et al., 2022). In this thesis, a modified algorithm for unconstrained convex optimization of inverse OT by Ma et al. (2020) will be used.

Once we have obtained a cost matrix, we can analyze the shape of the cost matrix to understand the dynamics in the simulations. Furthermore, different cost matrices for different simulations can quantify how the variations of parameters in the simulations affect the dynamics. Lastly, the cost matrix could highlight which physical processes or

initial conditions contribute most significantly to the formation of particular structures, such as clusters, filaments or voids.

In this thesis, we will first cover some theoretical foundations of OT and inverse OT in Chapter 2, whereafter we will describe the data we will use in more detail in Chapter 3. In Chapter 4, an algorithm is developed for inverse OT, with comments on the implementation and stability. In Chapter 5, the results of the algorithm are discussed for some reference cases, as well as for the FLAMINGO data.

2. Theoretical Foundations of Optimal Transport

Before we can work on the problem at hand, we need to properly define what (inverse) Optimal Transport (OT) is. For this, we will first recap a bit of measure theory.

The theory in this chapter is mainly based on Villani (2008), Benamou and Brenier (1998), Peyré and Cuturi (2019) and Chiu et al. (2022). To shorten this chapter, proofs of some statements are left out, they can be found in the mentioned papers.

2.1 Measure Theory

The reader is assumed to be familiar with the basic concepts of measure theory. Here, we will introduce a concept that is helpful to understand the language optimal transport theory is formulated in. We will look at the *coupling* of two measures:

Definition 2.1 (Coupling). *Let (X, μ) and (Y, ν) be two probability spaces. By constructing two random variables X, Y on some probability space $(\Omega, \mathbb{P})$, μ and ν are coupled if $\text{law}(X) = \mu$ and $\text{law}(Y) = \nu$, with the couple (X, Y) being called a **coupling** of (μ, ν) .*

Here, $\text{law}(X)$ is the probability distribution of X . By abuse of terminology, the law of (X, Y) is also called a coupling of (μ, ν) .

If μ and ν are the only laws relevant in the problem, we can assume, without loss of generality, $\Omega = X \times Y$. Hence, coupling μ and ν means constructing a measure π on $X \times Y$ such that π admits μ and ν as marginals on X and Y respectively. This measure π is what we later define as the transportation matrix.

Definition 2.2 (Deterministic coupling). *Using the same notation as in Definition 2.1, a coupling (X, Y) is called **deterministic** if there exists a measurable function $T : X \rightarrow Y$ such that $Y = T(X)$.*

This is equivalent to saying one of the following statements:

- (X, Y) is a coupling of μ and ν whose law π is such that Y can be expressed as a measurable function T on X almost everywhere.
- X has law μ and $Y = T(X)$ where $T_{\#}\mu = \nu$, with $\#$ the push forward.

In Definition 2.2, T is uniquely defined μ -almost surely, and is called the **transport map**.

2.2 Monge-Kantorovich Minimization Problem

Assume we have a pile of sand and a hole we have to fill with the sand. We can model the sand in both the pile and the hole by probability measures μ and ν , defined on some measure spaces X and Y . Let $A \subseteq X$ and $B \subseteq Y$ be measurable subsets of X , respectively Y . Then, $\mu(A)$ gives a measure of how much sand is located in A and $\nu(B)$ of how much sand is already in B . Moving the sand needs some effort, which can be modeled by a measurable *cost function* $c : X \times Y \rightarrow \mathbb{R} \cup \{+\infty\}$, where we assume the cost to move sand from A to B can be real-valued or infinite. One can then ask the following question, also known as the mass transport problem: how do we realize the transportation of the sand from A to B at minimal cost? This is the question [Monge \(1781\)](#) came up with in his *Mémoire sur la théorie des déblais et des remblais*, which led to the rise of the theory of optimal transport.

Example 2.3 (Monge-Kantorovich minimization problem). The **optimal coupling** or **optimal transport** is a coupling where one introduces a **cost function** $c(x, y)$ on $X \times Y$, that can be interpreted as the work needed to move one unit of mass from location x to location y . Then one considers the *Monge-Kantorovich minimization problem*

$$\inf_{(\mu, \nu) \sim (X, Y)} \mathbb{E} c(X, Y),$$

where the pair (X, Y) runs over all possible couplings of (μ, ν) , or equivalently

$$\inf_{\pi \in \Pi(\mu, \nu)} \int_{X \times Y} c(x, y) d\pi(x, y), \quad (2.1)$$

where the infimum runs over all joint probability measures π on $X \times Y$ with marginals μ and ν . Such joint measures are called *transference plans* (or transport plans, or transportation plans), and those achieving the infimum are called *optimal transference plans*.

Remark 2.4. The transference plans in Example 2.3 are couplings as in Definition 2.1, and they may be deterministic as in Definition 2.2. Hence, the problem of Optimal Transport can be rewritten into the question of looking for a (deterministic) optimal coupling.

Now, the problem of finding the optimal transport can be reformulated as follows: let ρ_0 and ρ_T be bounded, positive and measurable functions with compact support in $\mathbb{R}^d$. We require they integrate to the same number, which can be normalized to 1:

$$\int_{\mathbb{R}^d} \rho_0(x) dx = \int_{\mathbb{R}^d} \rho_T(x) dx = 1. \quad (2.2)$$

We then need to find a mapping, or *application*, $M : \mathbb{R}^d \rightarrow \mathbb{R}^d$ which realizes the transport from ρ_0 to ρ_T . In other words, we need to find an M for which for all $A \in \mathcal{B}(\mathbb{R}^d)$, there holds

$$\int_{M^{-1}(A)} \rho_0(x) dx = \int_A \rho_T(x) dx. \quad (2.3)$$

Here, M should be achieving the minimal cost of

$$\int_{\mathbb{R}^d} c(x, M(x)) \rho_0(x) dx. \quad (2.4)$$

Note that the condition in equation (2.3) is equivalent to

$$\int_{\mathbb{R}^d} f(M(x))\rho_0(x) dx = \int_{\mathbb{R}^d} f(x)\rho_T(x) dx \quad (2.5)$$

for all continuous functions f .

Example 2.5 (Cost function). Oftentimes, ρ_0 and ρ_T are the initial and final density distributions. Then, equation (2.2) is the requirement of conservation of mass, and the integrals are all in $\mathbb{R}^3$.

When ρ_0 and ρ_T describe density, the cost function c in equation (2.4) is a function that oftentimes is taken as

$$c : \mathbb{R}^3 \times \mathbb{R}^3 \rightarrow \mathbb{R}, \quad c(x, y) = \|x - y\|^r,$$

for a certain $r > 0$ (as "cost" should increase by distance), with $\|\cdot\|$ the Euclidean distance in $\mathbb{R}^3$.

A very common value of r for the cost function is $r = 2$ (Benamou and Brenier, 1998). This choice is often made because the squared Euclidean distance has applications in fluid dynamics and classical mechanics from the kinetic energy term (Benamou and Brenier, 2000).

Our problem then rewrites to finding the cost function $c : \mathbb{R}^3 \times \mathbb{R}^3 \rightarrow \mathbb{R}$ for which equation (2.4) is being minimized under our observed transport.

2.3 Discrete Optimal Transport

Now, let us look at discrete OT. For now, let us assume our problem has the same initial and final space, $X = Y = \mathbb{R}^n$. Given a cost matrix $c \in \mathbb{R}^{n \times n}$ with elements $c_{i,j}$, OT as in (2.3) can be rewritten as the search for a mapping M in the set $\text{Perm}(n)$ of permutations of n elements while solving

$$\min_{\sigma \in \text{Perm}(n)} \frac{1}{n} \sum_{i=1}^n c_{i, \sigma(i)}.$$

One could naively evaluate this function for all permutations, but the set of all permutations has size $n!$. Hence, for larger datasets, this is not computationally feasible.

Remark 2.6 (Uniqueness). Note that the optimal transport problem may have several optimal solutions. Consider, for example, $X = Y = \mathbb{R}^2$, with c given as the pairwise distance matrix between the corners of a 2-dimensional square of side length 1. In that case, only two assignments exists $\sigma = (1, 2)$ or $\sigma = (2, 1)$, and they are both optimal. These assignments can be seen in Figure 2.1.

Example 2.7 (Discrete Monge-Kantorovich minimization problem). Let $x_1, \dots, x_m \in X$ and $y_1, \dots, y_n \in Y$. Assume we have two discrete measures

$$\alpha = \sum_{i=1}^m a_i \delta_{x_i} \quad \text{and} \quad \beta = \sum_{j=1}^n b_j \delta_{y_j},$$

with weights $a \in \mathbb{R}^m$ and $b \in \mathbb{R}^n$ and positions x and y , with δ_x the Dirac measure at position x . Assume all elements of a and b are positive to avoid degeneracy issues. Then,

the Monge-Kantorovich minimization problem as in Example 2.3 rewrites to the search for a map $T : \{x_1, \dots, x_m\} \rightarrow \{y_1, \dots, y_n\}$ such that for all $j \in \{1, \dots, n\}$ there holds

$$b_j = \sum_{i: T(x_i)=y_j} a_i =: T_{\#}\alpha.$$

Note that, since we assumed all elements of b are positive, T is surjective.

Given a cost $c(x, y)$ defined for certain $(x, y) \in X \times Y$, T should minimize the transportation cost,

$$\min_T \left\{ \sum_i c(x_i, T(x_i)) : T_{\#}\alpha = \beta \right\}.$$

Again, such a map T can be encoded by a permutation σ . Then, the discrete version of equation (2.3) becomes

$$\sum_{i \in \sigma^{-1}(j)} a_i = b_j,$$

with $\sigma^{-1}(j)$ the pre-image set of j . Note that if $m = n$ and $a_i = b_j = 1/n$ (all the weights are uniform), then T is a bijection with $T(x_i) = y_{\sigma(j)}$ and the problem is equivalent to the optimal matching problem (2.3) with $c_{ij} := c(x_i, y_j)$.

Remark 2.8 (Existence). Note that a map T as in Example 2.7 may not always exist if $m \neq n$. For example, if $m < n$, the weight vectors are not compatible, and a map T from α to β can be found. However, a map from β to α cannot be found, since not all elements of α will be reached, and we require T to be surjective.

2.3.1 Solving the Monge-Kantorovich Minimization Problem

In general, OT can be solved with a linear program. The goal of linear programming is to solve optimization problems with linear objective functions and constraints, which is the case in OT (Peyré and Cuturi, 2019). In fact, optimal transport problems are equivalent to an important class of linear programs known as cost network flows (Korte and Vygen, 2018; L. R. Ford and Fulkerson, 1962).

Consider the following OT problem:

$$\min_{\pi \in \Pi(\alpha, \beta)} \sum_{i=1}^m \sum_{j=1}^n c_{ij} \pi_{ij}, \quad (2.6)$$

with $\Pi(a, b)$ the set of couplings on (α, β) and c a given cost function. One can rewrite equation (2.6) into a standard form. Let $\mathbb{1}_{m \times m}$ be the $m \times m$ identity matrix and let $a \otimes b$ be the Kronecker product. Then, the $(m+n) \times mn$ matrix

$$A = \begin{bmatrix} \mathbb{1}_{m \times m}^\top \otimes \mathbb{1}_{n \times n} \\ \mathbb{1}_{m \times m} \otimes \mathbb{1}_{n \times n}^\top \end{bmatrix}$$

can be used to encode the constraints: consider a matrix $P \in \mathbb{R}^{m \times n}$. One can flatten the matrix P onto a vector $p \in \mathbb{R}^{mn}$ by writing $p_{i+m(j-1)} = P_{ij}$. Then, one has (Peyré and Cuturi, 2019):

$$P \in \mathbb{R}^{m \times n} \in \Pi(\mu, \nu) \quad \Leftrightarrow \quad p \in \mathbb{R}_{\geq 0}^{mn}, \quad Ap = \begin{bmatrix} \mu \\ \nu \end{bmatrix}.$$

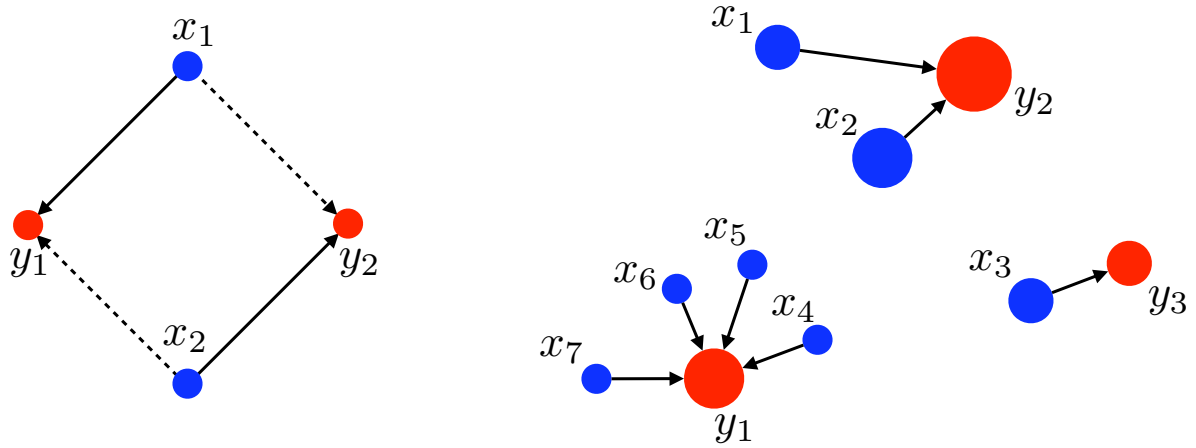


Figure 2.1: Left: blue dots from measure μ and red dots from measure ν . The dots are pairwise equidistant. Hence, either matching $\sigma = (1, 2)$ or $\sigma = (2, 1)$ is optimal.

Right: a map T can associate measure α to measure β , with weights proportional to the area of the disk. The mapping is as follows: $T(x_1) = T(x_2) = y_2$, $T(x_3) = y_3$ and $T(x_4) = T(x_5) = T(x_6) = T(x_7) = x_1$. However, there is no surjective T from β to α .

Figure 2.2 from [Peyré and Cuturi \(2019\)](#).

Hence, the original transport problem can be written as

$$\min_{p \in \mathbb{R}_{\geq 0}^{mn}} c^\top p, \quad \text{such that } Ap = \begin{bmatrix} \mu \\ \nu \end{bmatrix}.$$

However, note that $p \in \mathbb{R}^{mn}$, so we have $N = mn$ variables. Furthermore, A is a $(m + n) \times mn$ matrix, so we have $K = m + n$ equality constraints. In general, linear programs have a complexity of $\mathcal{O}(N^2 K)$ ([Vanderbei, 2020](#)), which is cubic complexity. This can cause issues for large values of m and n .

2.4 Entropy Regularization

To solve the computational difficulties in OT, [Cuturi \(2013\)](#) proposed an entropy regularization of OT. His approach smoothed the classical OT problem with an entropic term, given by

$$\pi = \operatorname{argmin}_{\pi \in \Pi(\mu, \nu)} \{ \langle c, \pi \rangle - \varepsilon H(\pi) \}, \quad (2.7)$$

with $\varepsilon > 0$, $\Pi(\mu, \nu)$ the set of all couplings between μ and ν , $\langle c, \pi \rangle$ the inner product between c and π , and

$$H(\pi) := - \sum_i \sum_j \pi_{ij} \log \pi_{ij} \quad (2.8)$$

the entropy of π , with π having entries π_{ij} . We use the convention $H(\pi) = -\infty$ if one of the entries of π is 0 or negative. The entropy regularized OT solves for the optimal coupling π that minimizes the (entropy regularized) cost of transfer from X to Y .

Note that for π a probability matrix, H is strongly concave: $\partial^2 H(\pi) = -\operatorname{diag}(1/\pi_{ij})$ and $\pi_{ij} \leq 1$. Therefore, the entropy-regularized problem has a unique solution.

Figure 2.2 shows the effect of the entropy regularization on the simplex Σ^3 , visualized as a triangle. Note how the entropy term pushes the original linear program solution away from the boundary of the triangle.

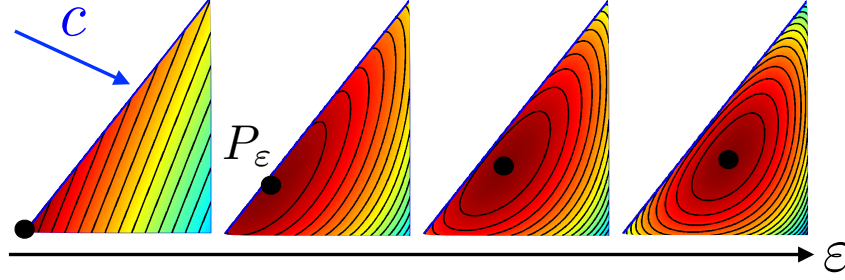


Figure 2.2: Impact of ε on the optimization of a linear function on the simplex Σ^3 , solving equation (2.7) for a varying ε . Figure 4.1 from [Peyré and Cuturi \(2019\)](#).

Entropy regularizing OT is helpful, because it takes into account the uncertainty and incompleteness of the data, and it is faster than regular OT. Furthermore, for $\varepsilon \rightarrow 0$, the entropy regularized OT will converge to the regular OT. For $\varepsilon \rightarrow \infty$, the solution converges to the coupling with maximal entropy between the two marginals μ and ν , $\mu\nu^\top$ ([Peyré and Cuturi, 2019](#)). Thus, as ε increases, we have faster computational algorithms and statistical convergence.

2.4.1 Sinkhorn Scaling

Now, let us look at a specific algorithm for solving equation (2.7). We can define the Kullback-Leibner (KL) divergence between two couplings π and $\hat{\pi}$ as follows:

$$\text{KL}(\hat{\pi}, \pi) := \sum_{i,j} \hat{\pi}_{ij} \log \left(\frac{\hat{\pi}_{ij}}{\pi_{ij}} \right). \quad (2.9)$$

Then, the (unique) solution of (2.7) is a projection onto $\Pi(\mu, \nu)$ of the Gibbs kernel associated to the cost matrix c as

$$K_{ij} := \exp(-c_{ij}/\varepsilon). \quad (2.10)$$

After all, when combining (2.8) with (2.7) and using $c_{ij}/\varepsilon = -\log K_{ij}$ from (2.10) we get

$$\begin{aligned} & \min_{\pi \in \Pi(\mu, \nu)} \sum_{i,j} \pi_{ij} c_{ij} + \varepsilon \sum_{i,j} \pi_{ij} \log \pi_{ij} \\ &= \min_{\pi \in \Pi(\mu, \nu)} \sum_{i,j} \frac{\pi_{ij} c_{ij}}{\varepsilon} + \sum_{i,j} \pi_{ij} \log \pi_{ij} \\ &= \min_{\pi \in \Pi(\mu, \nu)} - \sum_{i,j} \pi_{ij} \log K_{ij} + \sum_{i,j} \pi_{ij} \log \pi_{ij} \\ &= \min_{\pi \in \Pi(\mu, \nu)} \sum_{i,j} \pi_{ij} \log \frac{\pi_{ij}}{K_{ij}} \\ &= \text{KL}(\pi, K). \end{aligned}$$

Proposition 2.9. *The solution to (2.7) is unique and has the form*

$$\pi_{ij} = u_i K_{ij} v_j, \quad (2.11)$$

for $1 \leq i \leq m, 1 \leq j \leq n$ and two scaling variables $u \in \mathbb{R}_{\geq 0}^m, v \in \mathbb{R}_{\geq 0}^n$.

The proof can be found in [Peyré and Cuturi \(2019\)](#), proposition 4.3.

Note that equation (2.11) can be rewritten as $\pi = \text{diag}(u)K\text{diag}(v)$. Therefore, there must hold

$$\text{diag}(u)K\text{diag}(v)\mathbb{1}_n = \mu \quad \text{and} \quad \text{diag}(v)K^\top\text{diag}(u)\mathbb{1}_m = \nu. \quad (2.12)$$

Now, note that $\text{diag}(v)\mathbb{1}_n = v$ and $\text{diag}(u)\mathbb{1}_m = u$, thus we can see that this is equivalent to

$$u \otimes (Kv) = \mu \quad \text{and} \quad v \otimes (K^\top u) = \nu, \quad (2.13)$$

with $\otimes$ the entry-wise multiplication of vectors. This problem is known as the matrix scaling problem ([Nemirovski and Rothblum, 1999](#)).

One way to solve this problem is by Sinkhorn's algorithm ([Sinkhorn, 1964](#)), which can be seen in Algorithm 1. First we modify u so that it satisfies the left hand side of (2.13) and then modify v so that it satisfies the right hand side. The updates are then given by

$$u^{(l+1)} := \frac{\mu}{Kv^{(l)}} \quad \text{and} \quad v^{(l+1)} := \frac{\nu}{K^\top u^{(l+1)}}, \quad (2.14)$$

initialized by some arbitrary vector $v^{(0)} \in \mathbb{R}^n$. We stop the iterations when the Euclidean distance $\|\cdot\|^2$ between the updates of u and v is smaller than some convergence limit k_u and k_v respectively.

Remark 2.10. A different initialization will likely end with a different u and v , as they are only defined up to a multiplicative constant: if u and v satisfy (2.12), then λu and v/λ also satisfy (2.12), for any $\lambda > 0$. However, these iterations will converge and lead to the same optimal coupling ([Peyré and Cuturi, 2019](#)). Furthermore, note that Algorithm 1 is always convergent for large enough ε .

Example 2.11 (Sinkhorn's Algorithm). Consider $X = \{x_1, x_2\} = \{1, 3\}$ and $Y = \{y_1, y_2\} = \{2, 4\}$ and two observed probabilities $\mu = (\mu_1, \mu_2) = (1/3, 2/3)$ and $\nu = (\nu_1, \nu_2) = (2/3, 1/3)$. Assume we are given a cost function $c(x, y) = \|x - y\|^2$, with $\|\cdot\|^2$ the Euclidean distance. Then, we can see for the cost matrix:

$$c = \begin{pmatrix} (1-2)^2 & (1-4)^2 \\ (3-2)^2 & (3-4)^2 \end{pmatrix} = \begin{pmatrix} 1 & 9 \\ 1 & 1 \end{pmatrix}.$$

Algorithm 1: Sinkhorn's Algorithm for OT

Input: Cost matrix $c \in \mathbb{R}^{m \times n}$ and regularization parameter $\varepsilon > 0$ and marginals $\mu \in \mathbb{R}^m, \nu \in \mathbb{R}^n$.

Initialize: $v = (1/n, \dots, 1/n) \in \mathbb{R}^n$ and $K = \exp(c/\varepsilon)$

repeat

| $u \leftarrow \mu/(Kv)$;
| $v \leftarrow \nu/(K^\top u)$;

until convergent: $\|u^{(l)} - u^{(l+1)}\|^2 < k_u$ and $\|v^{(l)} - v^{(l+1)}\|^2 < k_v$;

Output: $\hat{\pi} = uKv$

Now, let us investigate Sinkhorn's algorithm. Initialize $v^0 = (1/2, 1/2)$ and choose $\varepsilon = 0.1$. We can see

$$K = e^{-c/\varepsilon} = \begin{pmatrix} e^{-1/0.1} & e^{-9/0.1} \\ e^{-1/0.1} & e^{-1/0.1} \end{pmatrix} \approx \begin{pmatrix} 4.5 \cdot 10^{-5} & 8.2 \cdot 10^{-40} \\ 4.5 \cdot 10^{-5} & 4.5 \cdot 10^{-5} \end{pmatrix}.$$

Let us now look at the first iteration:

$$u^{(1)} = \frac{\mu}{Kv^{(0)}} = \begin{pmatrix} 14684.3 \\ 14684.3 \end{pmatrix}, \quad \text{and} \quad v^{(1)} := \frac{\nu}{K^\top u^{(1)}} = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix}.$$

Let us also calculate the second iteration:

$$u^{(2)} = \frac{\mu}{Kv^{(1)}} = \begin{pmatrix} 14684.3 \\ 14684.3 \end{pmatrix}, \quad \text{and} \quad v^{(2)} := \frac{\nu}{K^\top u^{(2)}} = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix}.$$

If we calculate the Frobenius norm, $\|\cdot\|_F$ of these two iterations, we can see

$$\|u^{(2)} - u^{(1)}\|_F \approx 0, \quad \text{and} \quad \|v^{(2)} - v^{(1)}\|_F \approx 0.$$

This is a very small deviation, and indeed, if we calculate even more iterations, we can see that difference between $u^{(l)}$ and $u^{(l+1)}$ and the difference between $v^{(l)}$ and $v^{(l+1)}$ gets smaller and smaller, thus the algorithm is converging.

At iteration 20, the difference in u stays down at 0, as well as the difference in v . Calculating the transportation matrix $\hat{\pi}$ from this, we get

$$\hat{\pi} = u_i v_j K_{ij} \approx \begin{pmatrix} 0.33 & 6.02 \cdot 10^{-36} \\ 0.33 & 0.33 \end{pmatrix}.$$

2.5 Inverse Optimal Transport

In some cases, you already know the transportation matrix π , but you do not know the cost matrix c yet. This case, where you want to solve for c , is called *inverse Optimal Transport*, or iOT; it studies the problem of inferring the underlying cost matrix given an observation on coupling two probability measures, such as in transportation.

To solve for the cost function, we will first need to define iOT. However, unlike for the forward problem (Villani, 2008), the mathematical foundations of the inverse problem have not been intensively studied. Some of the main contributions on iOT are applications of internal migration (Stuart and Wolfram, 2020), estimating marriage matching (Ma et al., 2020), the problem of iOT with noisy and incomplete matching data (Li et al., 2019), and the definition on discrete probabilistic iOT (Chiu et al., 2022).

To define iOT, let us look at two measurable spaces $X = \{x_1, \dots, x_m\}$ and $Y = \{y_1, \dots, y_n\}$, with $m, n \geq 2$, as the cases where $m = 1$ or $n = 1$ are trivial. Let $\pi = (\pi_{ij})_{m \times n} \in \mathbb{R}_{\geq 0}^{m \times n}$ be a coupling. Here, π_{ij} represents the probability of observing x_i and y_j simultaneously. This interaction can be obtained by a cost matrix $c = (c_{ij})_{m \times n} \in (\mathbb{R}_{\geq 0} \cup \{\infty\})^{m \times n}$, with c_{ij} being the cost of coupling x_i and y_j , just as in the forward problem.

Now, given a coupling π , the inference on c can be formulated as an inverse entropy regularized OT. We know that entropy regularized OT is the map

$$\Phi : (\mathbb{R}_{\geq 0} \cup \{\infty\})^{m \times n} \times \mathcal{P}(\mathbb{R}^m) \times \mathcal{P}(\mathbb{R}^n) \rightarrow \mathcal{P}(\mathbb{R}_{\geq 0}^{m \times n}),$$

with $\Phi(c, \mu, \nu) = \pi$, π as in (2.7) and $\mathcal{P}(X)$ the set of probability measures on X . Then, the inverse entropy regularized OT can be defined as $\Phi^{-1} : \pi \rightarrow \{(c, \mu, \nu)\}$, with $\{(c, \mu, \nu)\}$ the pre-image set of π under Φ .

Remark 2.12. If π is observed without any noise, it completely determines μ and ν : from $(c, \mu, \nu) \in \Phi^{-1}(\pi)$, we know $\pi \in \Pi(\mu, \nu)$, thus μ and ν are the marginals of π . Thus, we can reduce the problem into $\Phi^{-1} : \pi \rightarrow \{c\}$. However, $\Phi^{-1} : \pi \rightarrow \{c\}$ is not a well-defined function, since π does not uniquely determine c ; for example two cost matrices may differ by an additive constant.

Note that we are looking at the entropy regularized OT instead of the regular OT, since the transportation matrix $\hat{\pi}$ is in general sparse, and zero elements do not carry enough information to say something meaningful about the cost: a zero in $\hat{\pi}$ does not necessarily mean that the underlying cost is infinite (Chiu et al., 2022).

3. *Data*

3.1 *FLAMINGO*

The main data used is the FLAMINGO (Full-hydro Large-scale structure simulations with All-sky Mapping for the Interpretation of Next Generation Observations) simulations, see also [Schaye et al. \(2023\)](#); [Kugel et al. \(2023\)](#). The FLAMINGO simulations are large-scale hydrodynamical simulations, that explicitly incorporate baryonic physics. The aim of these simulations is to provide more accurate theoretical predictions for observational cosmology, particularly on smaller scales where baryonic effects become more dominant and complex.

The FLAMINGO data we used contains an initial distribution of particles, and eight subsequent evolutionary timestamps. In general, information includes the scale factor and redshifts. Then, for each timestamp, information about the particle locations, types and masses are given.

We used a dark-matter only simulation, with a box size of 400 Mpc and 360^3 particles. A projection of the cold dark matter surface density can be found in Figure 3.1. Some denser and less-dense structures can be seen, as we would expect in the Cosmic Web. The data was processed with `swiftsimio` ([Borrow and Borrisov, 2020](#)). Besides the FLAMINGO data, we have generated data for specific patterns of motion, which will be elaborated in Chapter 5.

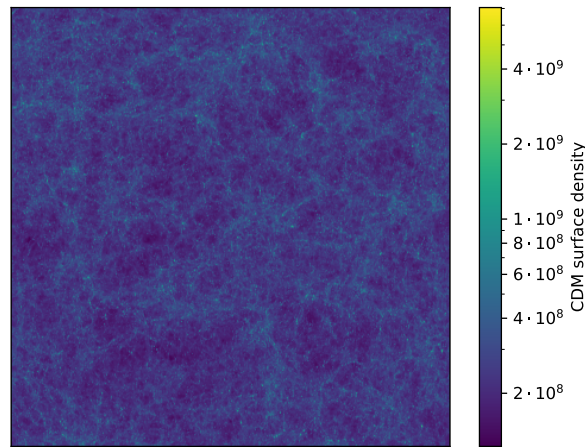


Figure 3.1: Cold Dark Matter (CDM) surface density of the used FLAMINGO simulation; a dark-matter only simulation with a box size of 400 Mpc and 360^3 particles.

3.1.1 Boxes

As the simulations have about 50 million particles for each time-step, evaluating each particle can be computationally expensive. To not look into each particle separately, the data is divided into n^3 boxes of equal size - each axis is divided into n parts. The boxes are counted in the following way: box 0 is the box around $(0, 0, 0)$. Then, taking the same x and y , we will count the boxes upward in z . Once we have reached the maximum z -value, we will move up one y -segment, and again count up in z . We repeat this until there are no more y -segments. Once this is the case, we move up one x -segment, and repeat this process. We stop once all the x -values have been covered, resulting in n^3 boxes. The $n = 2$ case can be seen in Figure 3.2. Note that we implicitly assume here, that there are no negative coordinates.

Division of data into boxes, $n = 2$

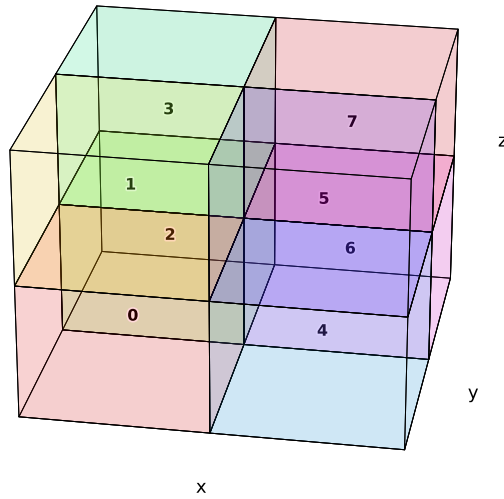


Figure 3.2: Division of the data into boxes, for the case $n = 2$.

By dividing the data into boxes, the cost matrix will be a function of these boxes, and not of the specific particles. Therefore, the cost matrix depends on the amount of particles in each box and the dynamics of those particles on average, rather than the dynamics of the individual particles.

4. Algorithm for Inverse Optimal Transport

Our objective is to find the cost matrix of the observed transport in our data. To find the cost matrix, we will use an algorithm inspired by [Ma et al. \(2020\)](#). We will follow part of their derivation, while discretizing some steps along the way. As we are working in a discrete case, all our input and output variables will be vectors and matrices. Let $n \in \mathbb{Z}_{\geq 1}$ be the number of points or, in our case, the number of boxes as described in Section 3.1.1. Our input and output spaces are the same space, $X \subseteq \mathbb{R}^m$ and $Y \subseteq \mathbb{R}^n$, with $m = n$. However, we will keep m and n separate for clarification in the derivations. Then, $\mu \in \Delta^{m-1}, \nu \in \Delta^{n-1}$ are probability vectors, with $\Delta^{n-1} := \{x = (x_1, \dots, x_n) \in \mathbb{R}^n : x_i \geq 0, \sum_{i=1}^n x_i = 1\}$ the standard probability simplex in $\mathbb{R}^n$, and $\pi, c \in \mathbb{R}^{m \times n}$. Note that μ and ν are (discrete) measures, but we can view them as vectors as we can assume all our data points are distinct and fixed with the same weight. With this notation, equation (2.7) can be written as

$$\min_{\pi \in \mathbb{R}^{m \times n}} \left\{ \langle c, \pi \rangle : \pi \geq 0, \pi \mathbb{1}_n = \mu, \pi^\top \mathbb{1}_m = \nu \right\}, \quad (4.1)$$

with $\langle c, \pi \rangle = \sum_{i,j} c_{ij} \pi_{ij}$ the inner product and $\mathbb{1}_n := (1, \dots, 1)^\top \in \mathbb{R}^n$, as before.

A modification of equation (4.1) was proposed by [Cuturi \(2013\)](#), with an additional entropy regularization term in the objective function, as in Section 2.4:

$$\min_{\pi \in \mathbb{R}^{m \times n}} \left\{ \langle c, \pi \rangle - \varepsilon H(\pi) : \pi \mathbb{1}_n = \mu, \pi^\top \mathbb{1}_m = \nu \right\}, \quad (4.2)$$

where $H(\pi) := -\langle \pi, \log \pi \rangle = -\sum_{i,j} \pi_{ij} (\log \pi_{ij})$ is the Shannon entropy of π , as in equation (2.8), and $\varepsilon > 0$ is the weight of the entropy regularization. Note that there are different definitions of H , some have an extra term -1 . However, this extra term does not matter for our purposes, as long as we are consistent with our definition of H . Due to the entropy term in (4.2), the inequality constraint in equation (4.1) is eliminated. Furthermore, the objective function in equation (4.2) becomes strictly convex, therefore having a unique solution ([Ma et al., 2020](#)).

For the rest of this chapter, let us write c for the theoretical cost matrix, c^* for the actual cost matrix and $\hat{c}$ for the cost matrix as retrieved from an algorithm.

4.1 The Dual Formulation of Inverse Optimal Transport

Now, let $\Pi(\mu, \nu)$ be the set of probability measures $\pi \in P(X \times Y)$ such that $\pi(A \times Y) = \mu(A)$ and $\pi(X \times B) = \nu(B)$, for any measurable set $A \subseteq X, B \subseteq Y$. Thus $\mu_i = \sum_j \pi_{ij}$ and $\nu_j = \sum_i \pi_{ij}$. Let $\alpha \in \mathbb{R}^m$ and $\beta \in \mathbb{R}^n$ be the Lagrangian multipliers corresponding to the marginal constraints. We can then rewrite equation (4.2) as

$$\min_{\pi \in \Pi(\mu, \nu)} \{ \langle c, \pi \rangle - \varepsilon H(\pi) \}. \quad (4.3)$$

Now, the dual problem of equation (4.3) can be calculated.

Lemma 4.1. *The dual problem of (4.3) is given by*

$$D(\alpha, \beta) := \sum_{i=1}^m \alpha_i \mu_i + \sum_{j=1}^n \beta_j \nu_j - \varepsilon \sum_{i=1}^m \sum_{j=1}^n \exp \left(\frac{\alpha_i + \beta_j - c_{ij} - \varepsilon}{\varepsilon} \right), \quad (4.4)$$

and the dual optimization problem is given by

$$\max_{\alpha \in \mathbb{R}^m, \beta \in \mathbb{R}^n} D(\alpha, \beta). \quad (4.5)$$

The proof of this lemma can be found in Appendix A.1.

Let (α^*, β^*) be the optimal solution for the dual problem. Note that from the proof of Lemma 4.1, equation (A.3) holds:

$$\pi_{ij}^* = \exp \left[\frac{\alpha_i^* + \beta_j^* - c_{ij}^* - \varepsilon}{\varepsilon} \right]. \quad (4.6)$$

Now, let us consider the inverse problem: suppose we are given $\mu \in \mathbb{R}^m, \nu \in \mathbb{R}^n$ and $\pi^* \in \mathbb{R}^{m \times n}$. We then want to retrieve the cost matrix $\hat{c} \in \mathbb{R}^{m \times n}$. (Ma et al., 2020) proposed an iOT to find $\hat{c}$, which is, in the discretized form:

$$\hat{c} = \min_c \text{KL}(\pi^*, \pi^c) + \varepsilon^{-1} R(c), \quad (4.7a)$$

$$\text{such that } \pi^c = \operatorname{argmin}_{\pi \in \mathbb{R}^{m \times n}} \left\{ \sum_{i,j} c_{ij} \pi_{ij} - \varepsilon H(\pi) \right\}. \quad (4.7b)$$

Here, $\text{KL}(\pi^*, \pi^c)$ is the (discrete) Kullback-Leibler (KL) divergence between π^* and π^c , as in equation (2.9) and $R(c)$ the regularization on c , which will be described more in Section 4.3.2. In essence, equations (4.7a) and (4.7b) adjust $\hat{c}$ so that the theoretical π^c , the optimal transport plan induced by c , is as close to the empirical π^* as possible, measured by the KL distance.

Note that the optimization problem in equations (4.7a) and (4.7b) is a bi-level optimization problem, which is usually quite computationally heavy. However, we can prove it is equivalent to an unconstrained problem in c , by using the dual form (4.5).

Theorem 4.2. *The optimization problem in equations (4.7a) and (4.7b) is equivalent to*

$$\min_{\alpha \in \mathbb{R}^m, \beta \in \mathbb{R}^n, c \in \mathbb{R}^{m \times n}} E(\alpha, \beta, c) + R(c), \quad (4.8)$$

with

$$E(\alpha, \beta, c) := \sum_{i,j} c_{ij} \hat{\pi}_{ij} - \sum_i \alpha_i \mu_i - \sum_j \beta_j \nu_j + \varepsilon \sum_{i,j} e^{(\alpha_i + \beta_j - c_{ij} - \varepsilon)/\varepsilon}. \quad (4.9)$$

Here, equivalent means that c^* solves (4.7) if and only if $(\alpha^{c^*}, \beta^{c^*}, c^*)$ solves (4.8) with $(\alpha^{c^*}, \beta^{c^*})$ the optimal solution for the dual problem (4.5) with cost c^* .

The proof can be found in Appendix A.2.

4.1.1 Equivalence of Solutions

The possibility of solving (4.8) more efficiently than (4.7), comes from the fact that E is a jointly convex functional on

$$\mathcal{W} := \mathbb{R}^m \times \mathbb{R}^n \times \mathbb{R}^{m \times n}.$$

Note, however, that E is not strictly convex in (α, β, c) , so we cannot claim uniqueness of the minimizer - there are actually infinitely many solutions of (4.8) with the same $\hat{\pi}$, if there is no additional constraint R (Ma et al., 2020). However, we can see that the optimal solution set is given by an equivalence class of some equivalence relation.

Definition 4.3. (α, β, c) and $(\bar{\alpha}, \bar{\beta}, \bar{c})$ are **equivalent**, $(\alpha, \beta, c) \sim (\bar{\alpha}, \bar{\beta}, \bar{c})$, if $\alpha_i + \beta_j - c_{ij} = \bar{\alpha}_i + \bar{\beta}_j - \bar{c}_{ij}$ for all $1 \leq i \leq m, 1 \leq j \leq n$.

It is easy to see that $\sim$ is an equivalence relation over $\mathcal{W}$. We will omit the proof for conciseness. The equivalence relation in Definition 4.3 induces a quotient space $\widetilde{\mathcal{W}} := \mathcal{W} / \sim$. Let $[(\alpha, \beta, c)] \subseteq \widetilde{\mathcal{W}}$ be the equivalence class of (α, β, c) . Then, there holds the following for E :

Theorem 4.4. *The following statements are true for $E(\alpha, \beta, c)$ in (4.8):*

- (i) $E(\alpha, \beta, c)$ is jointly convex in (α, β, c) ;
- (ii) E is a constant on the equivalence class $[(\alpha, \beta, c)]$ for each $(\alpha, \beta, c) \in \mathcal{W}$;
- (iii) The functional $\widetilde{E} : \widetilde{\mathcal{W}} \rightarrow \mathbb{R}$ with $\widetilde{E}([(\alpha, \beta, c)]) = E(\alpha, \beta, c)$ is well defined;
- (iv) If (α^*, β^*, c^*) is a solution of (4.8), then the optimal transport plan π^* in (4.7) is given by

$$\pi_{ij}^* = e^{(\alpha_i^* + \beta_j^* - c_{ij}^* - \varepsilon)/\varepsilon};$$

- (v) $[(\alpha^*, \beta^*, c^*)]$ as in (iv) is the unique minimizer of $\widetilde{E}$.

The proof can be found in Appendix A.3.

From Theorem 4.4 we can see that (4.8) is convex as long as R is convex, or imposes constraints on a convex set. But even without knowledge of a specific R , we can characterize the entire optimal solution set, by finding one minimizer of E and making use of the equivalence relation.

4.2 Algorithm Development

As we have seen, π is a real-valued $m \times n$ matrix. In our case, we can divide the data into boxes, as described in Section 3.1.1, and define

$$\hat{\pi}_{ij} := \frac{N_{ij}}{N},$$

with N_{ij} the number of particles going from box i to box j , and $N = \sum_{i,j} N_{ij}$. Then, we have the following optimization problem of equation (4.8):

$$\min_{\alpha \in \mathbb{R}^m, \beta \in \mathbb{R}^n, \hat{c} \in \mathbb{R}^{m \times n}} \{R(\hat{c}) - \langle \alpha, \mu \rangle - \langle \beta, \nu \rangle + \langle \hat{c}, \hat{\pi} \rangle + s(\alpha, \beta, \hat{c})\}, \quad (4.10)$$

with $s : \mathbb{R}^m \times \mathbb{R}^n \times \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$ given as

$$s(\alpha, \beta, \hat{c}) := \varepsilon \sum_{i,j} e^{(\alpha_i + \beta_j - \hat{c}_{ij} - \varepsilon)/\varepsilon} = \varepsilon \langle e^{(\alpha_i + \beta_j - \hat{c}_{ij} - \varepsilon)/\varepsilon}, 1 \rangle. \quad (4.11)$$

To solve (4.10), we can use a update scheme also known as Block Coordinate Descent (BCD). In general, BCD requires joint convexity of E , which is shown to be true in Theorem 4.4, and Lipschitz continuity of its gradient (Beck, 2015), which is not true in our case. There is, however, an alternative formulation which is equivalent to the minimization in equation (4.8), where the objective function does have a Lipschitz continuous gradient.

Lemma 4.5. *The inverse Optimal Transport minimization (4.8) has the same set of solutions as the following minimization:*

$$\min_{\alpha, \beta, c} \Psi(\alpha, \beta, c) := F(\alpha, \beta, c) + R(c), \quad (4.12)$$

with

$$F(\alpha, \beta, c) = -\langle \alpha, \mu \rangle - \langle \beta, \nu \rangle + \langle c, \hat{\pi} \rangle + \varepsilon \log \left(\langle e^{(\alpha_i + \beta_j - c_{ij} - \varepsilon)/\varepsilon}, 1 \rangle \right).$$

The proof of this lemma can be found in Appendix A.4.

At this point, we can consider our Algorithm 2. We know that given c , the optimal coupling $\hat{\pi}$ between μ and ν as in (2.7) can be solved by applying (μ, ν) -Sinkhorn scaling on $K = e^{-\hat{c}/\varepsilon}$, as in Section 2.4.1. Ma et al. (2020) advocated for a modified algorithm to the standard BCD, with similar updates for α and β , but a modification for the update of $\hat{c}$ by splitting it into multiple steps for the inverse problem, as can be seen in Algorithm 2. After the Sinkhorn steps, as in Algorithm 1, we do a scaling to compute the $m \times n$ matrix K , with

$$K_{ij} = \frac{\hat{\pi}_{ij}}{e^{(\alpha_i + \beta_j)/\varepsilon}}, \quad (4.13)$$

for $i \in \{1, \dots, m\}, j \in \{1, \dots, n\}$. After all, $K = e^{-\hat{c}/\varepsilon}$ and, from (4.6)

$$\hat{\pi} = \exp \left(\frac{\alpha + \beta - \hat{c} - \varepsilon}{\varepsilon} \right).$$

Then, you take a proximal gradient descent to obtain $\hat{c}$:

$$\hat{c} = \text{prox}_{\gamma R}(-\varepsilon(\log(K) + 1)) := \operatorname{argmin}_{c \in \mathbb{R}^{m \times n}} \left\{ R(c) + \frac{1}{2\gamma} \|c - \xi\|^2 \right\}. \quad (4.14)$$

Algorithm 2: Cost Learning Algorithm for Inverse OT**Input:** Observed matrix $\hat{\pi} \in \mathbb{R}^{m \times n}$ and its marginals $\mu \in \mathbb{R}^m$, $\nu \in \mathbb{R}^n$ **Initialize:** $\alpha \in \mathbb{R}^{m \times 1}$, $\beta \in \mathbb{R}^{n \times 1}$, $u = \exp(\alpha/\varepsilon)$, $v = \exp(\beta/\varepsilon)$, $\hat{c} \in \mathbb{R}^{m \times n}$ **repeat**

$$\begin{aligned} & K \leftarrow e^{-\hat{c}/\varepsilon}; \\ & u \leftarrow \mu/(Kv); \\ & v \leftarrow \nu/(K^\top u); \\ & K \leftarrow \hat{\pi}/(uv^\top); \\ & c \leftarrow \text{prox}_{\gamma R}(-\varepsilon(\log(K) + 1)); \end{aligned}$$
until convergent: $\|u^{(l)} - u^{(l+1)}\|^2 < k_u$ and $\|v^{(l)} - v^{(l+1)}\|^2 < k_v$;**Output:** $\alpha = \varepsilon \log u$, $\beta = \varepsilon \log v$, $\hat{c}$

Here, $\xi := -\varepsilon(\log(K) + 1)$, $\|\cdot\|^2$ is the Frobenius norm and all operations are performed component-wisely. We stop when the Euclidean distance $\|\cdot\|^2$ between the updates of u and v is smaller than some convergence limit k_u and k_v respectively.

Remark 4.6. Note that, since $\hat{\pi}$ only has information about the ratio $\hat{c}/\varepsilon$, we can set $\varepsilon = 1$ without loss of generality. Then, we will recover $\hat{c}/\varepsilon$, which is c up to a constant scaling.

Remark 4.7. Notably, equation (4.13) is different than in Ma et al. (2020), as they have obtained a transport plan that is slightly different than ours,

$$\pi_{ij}^* = \exp \left[\frac{\alpha_i^* + \beta_j^* - c_{ij}^*}{\varepsilon} \right].$$

However, the extra $-\varepsilon$ term in our case was due to the chain rule in the proof of Lemma 4.1. This extra ε -term also alters the algorithm a bit from the algorithm of Ma et al. (2020). However, a constant scaling is not thought to matter, since α and β are probability vectors and $\hat{\pi}$ is a probability matrix, so extra scaling will factor out with the normalization of α, β and $\hat{\pi}$. As can be seen in Appendix B, the original algorithm of Ma et al. (2020) differs with a constant scaling of our algorithm, except for the M_c terms, which are a result of our choice of R .

4.3 Algorithm Implementation

Now, we can use Algorithm 2 to find the cost matrix of multiple different scenarios. In this section, we will comment on the implementation of the algorithm, whereas the results will be described in Chapter 5.

In the application of Algorithm 2, a convergence limit of $k_u = k_v = 10^{-4}$ was used.

4.3.1 Numerical Stability

Algorithm 2 is implemented in Python. For initializing α, β and $\hat{c}$, we have chosen a random initial conditions using `numpy`; `alpha = np.random.rand(n)`, `beta = np.random.rand(n)` and `c = np.random.rand(n, n)`, with n the number of boxes. For testing purposes, we also used these random initial conditions multiplied by 100. After running the algorithm

multiple times on the same dataset, we confirmed that the output of the algorithm is indeed independent of the initialization.

Floor

Note that $\hat{\pi}$ can contain quite a few zeros. For example, if we consider a simulation with only collapse towards the center point, we would not expect any movement from the center point outwards, which would result in zeros in $\hat{\pi}$. Due to the fact that in Algorithm 2 we set $K = \hat{\pi}/(uv^\top)$, and later divide by both Kv and $K^\top u$, if $\hat{\pi}$ has zero values, we will eventually end up dividing by 0. To solve this division problem, we manually changed the zero-values in μ, ν and $\hat{\pi}$ to a floor value, $1/N^3$, with N the number of particles. We expect this change to not have a significant influence on the generated cost matrix, because if we slightly increase the floor value, it does not change the cost matrix significantly for smaller values of n (the number of boxes). However, for larger n , a floor-value of $1/N$ will generate unexpected patterns in the cost matrix for the reference cases. These are thought to be artefacts of the floor and go away if we decrease the floor to $1/N^3$. After changing the values of μ, ν and $\hat{\pi}$ slightly, we re-normalize to ensure they are probability vectors.

4.3.2 Minimization and choosing $R(c)$

At the end of the loop of Algorithm 2, we need to calculate the proximal gradient for the minimization. This proximal gradient can cause some problems, as for a larger number of boxes n , a standard Python minimization package, such as `scipy.optimize.minimize`, usually creates a $n^2 \times n^2$ matrix to minimize the given expression, subsequently trying to create arrays which were too big for a personal computer. To solve these computational problems, we tried other minimization libraries, such as `cvxpy`, but this ended up having the same problem.

However, noted that at this point in the algorithm, we have α, β fixed. Let us first consider the problem (4.10) without $R(c)$:

$$\min_{c \in \mathbb{R}^{m \times n}} \left\{ \langle c, \hat{\pi} \rangle + \varepsilon \sum_{i,j} e^{(\alpha_i + \beta_j - c_{ij})/\varepsilon} \right\}.$$

To find the minimum, we can differentiate with respect to c and set the derivative to 0. This gives us, with the definitions $u = \exp(\alpha/\varepsilon)$ and $v = \exp(\beta/\varepsilon)$,

$$\hat{c}_{ij} = -\varepsilon \left[\log \left(\frac{\hat{\pi}_{ij}}{u_i v_j} \right) + 1 \right].$$

This is the minimum for all $R(c) \equiv 0$. If we now re-introduce $R(c)$, Ma et al. (2020) propose a change in the Sinkhorn algorithm by taking the proximal operator to obtain the updated c ,

$$\hat{c} = \text{prox}_{\gamma R}(\xi) := \underset{c \in \mathbb{R}^{m \times n}}{\text{argmin}} \left\{ R(c) + \frac{1}{2\gamma} \|c - \xi\|_2^2 \right\}.$$

Often, γ is taken to be 1. Note from Remark 4.7 that Ma et al. (2020) use $\xi = -\varepsilon \log K$, while we use $\xi = -\varepsilon(\log(K) + 1)$. Furthermore, note that the inverse OT formulation (4.8) is convex as long as $R(c)$ is convex or imposes a constraint onto a convex set (Ma

et al., 2020). Furthermore, from Theorem 4.4 we know that (4.8) has a unique equivalence set that minimizes E . Thus, even if we take $R(c)$ arbitrary, we can still characterize the entire optimal solution set using one minimizer of E . For example, if (α, β, c) minimizes E , then any other $(\bar{\alpha}, \bar{\beta}, \bar{c})$ will minimize E if and only if $(\alpha, \beta, c) \sim (\bar{\alpha}, \bar{\beta}, \bar{c})$.

Note that if we choose a certain $R(c)$ for all our applications, we can compare the results without having to worry about different equivalence classes.

Remark 4.8 (Projection Operator). Note that $\text{prox}_{\gamma R}(\xi)$ is the generalization of the projection operator. Choose $R = \chi_U : \mathbb{R}^{m \times n} \rightarrow \mathbb{R} \cup \{\infty\}$ for some closed, convex, non-empty $U \subset \mathbb{R}^{m \times n}$ given by

$$\chi_U(x) = \begin{cases} 0, & x \in U; \\ +\infty, & x \notin U. \end{cases}$$

We can see

$$\begin{aligned} \text{prox}_{\gamma R}(\hat{c}) &= \underset{c \in U}{\operatorname{argmin}} \begin{cases} \frac{1}{2\gamma} \|c - \hat{c}\|_2^2, & c \in U; \\ +\infty, & c \notin U; \end{cases} \\ &= \underset{c \in U}{\operatorname{argmin}} \frac{1}{2\gamma} \|c - \hat{c}\|_2^2. \end{aligned}$$

As this projection is easily calculated, we chose for R the case $R(c) = \chi_U(c)$ for some $U \subseteq \mathbb{R}^{m \times n}$, so $\text{prox}_{\gamma R}(\xi)$ is the projection operator. If we now choose $U = [0, M_c]^{m \times n}$, we can see

$$\hat{c}_{ij} = \begin{cases} 0, & \hat{c}_{ij} < 0; \\ \hat{c}_{ij}, & 0 \leq \hat{c}_{ij} < M_c; \\ M_c, & \hat{c}_{ij} \geq M_c. \end{cases}$$

This choice of $R(c)$ is easily implementable using `np.clip`. Furthermore, due to the fact that all values of c which are higher than M_c get clipped back to M_c , which improves numerical stability by implementing a ceiling on how high entries of c can go.

5. Recovering the Cost Matrix

In this Chapter, we will use Algorithm 2 to find the cost matrix of the simulations. As described in Section 4.3, $R(c) = \mathbb{1}_U$ and $U = [0, M_c]^3$ are used. For all future results, we choose $\gamma = \varepsilon = 1$. Note that we will only recover c up to a constant scaling, and not c itself; we will recover c/ε . Therefore, the value of ε we choose is not important for the results, as long as a consistent value of ε is used. Furthermore, a value of $M_c = 10$ is chosen, in order to have enough accuracy and still be computationally efficient enough to run multiple cases.

5.1 Reference Cases

To help interpret the results of FLAMINGO data, a few reference cases will be calculated. To better see differences in the heat map of the acquired cost matrix, we manually set all entries that were equal to M_c , equal to the highest entry of c that is not equal to M_c . In some cases, this scaling will not be performed as this will result in a uniform heat map. These cases will be mentioned separately. All reference cases are simulated with one million particles. In this section, some initial reference cases were tested for a certain number of boxes. More reference cases are described in Appendix C.

5.1.1 Linear Motion

As a first reference case, consider particles with linear motion and constant velocity. For the simulation, a box size of 100 (arbitrary units) was used. All particles started in the box $[0, 50]^3$, and moved with the same velocity towards $[50, 100]^3$. All particles have an equal x -, y - and z -velocity.

For the case $n = 2$, where each axis is divided into 2 segments, we have 8 boxes in total. Here, a low cost for c_{07} is expected, and a high cost everywhere else. This low cost at c_{07} is because all particles are going from box 0 (which is $[0, 50]^3$) to box 7 (which is $[50, 100]^3$), and no particles are going anywhere else. As we can see in Figure 5.1, in the case $n = 2$ we have indeed a very low cost in c_{07} and a high cost everywhere else. Note that in Figure 5.1, and all following heat maps of the cost matrix, the y -axis is the i in c_{ij} , where the particles came from, and the x -axis is the j in c_{ij} , where the particles are moving to. Note that the $n = 2$ case in Figure 5.1 is not scaled as described above in Section 5.1, as this scaling would result in a uniform heat map.

Also for $n = 4$ we can see a structure we would expect - there is some motion from $[0, 50]^3$, which is now divided into multiple boxes with low box number, to $[50, 100]^3$, which is divided into boxes with a higher box number.

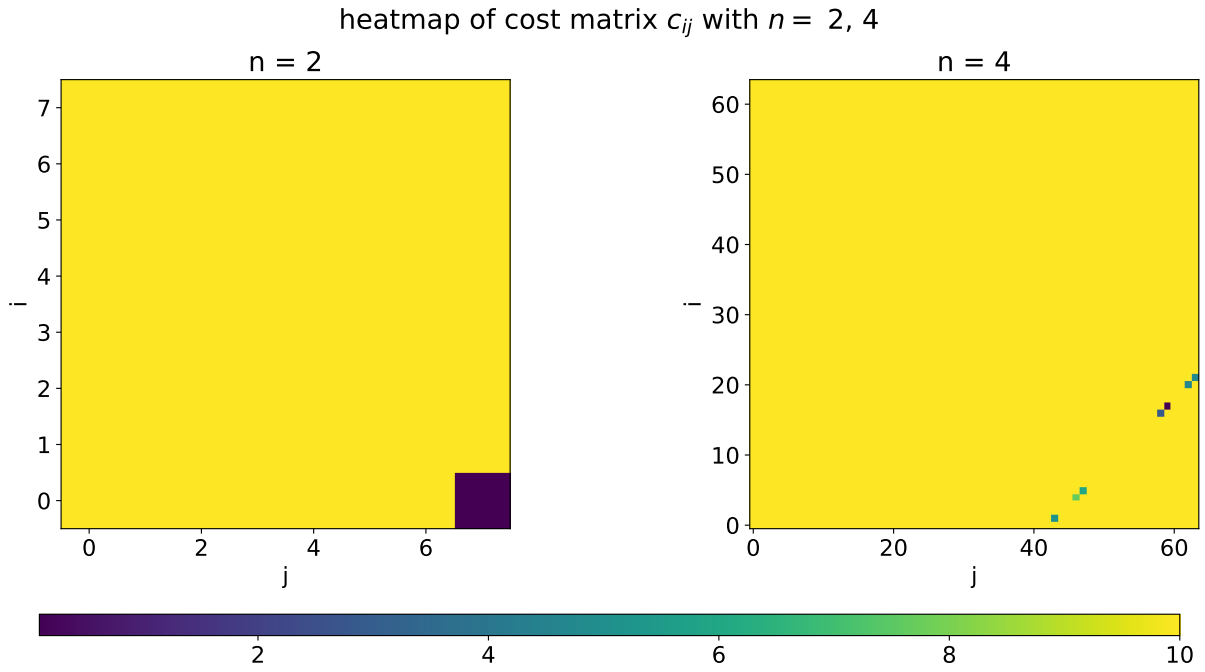


Figure 5.1: Heat map of the cost matrix for uniform linear motion from $[0, 50]^3$ to $[50, 100]^3$, for $n = 2$ (left) and $n = 4$ (right), as retrieved from Algorithm 2. The $n = 2$ case is not scaled as described in Section 5.1.

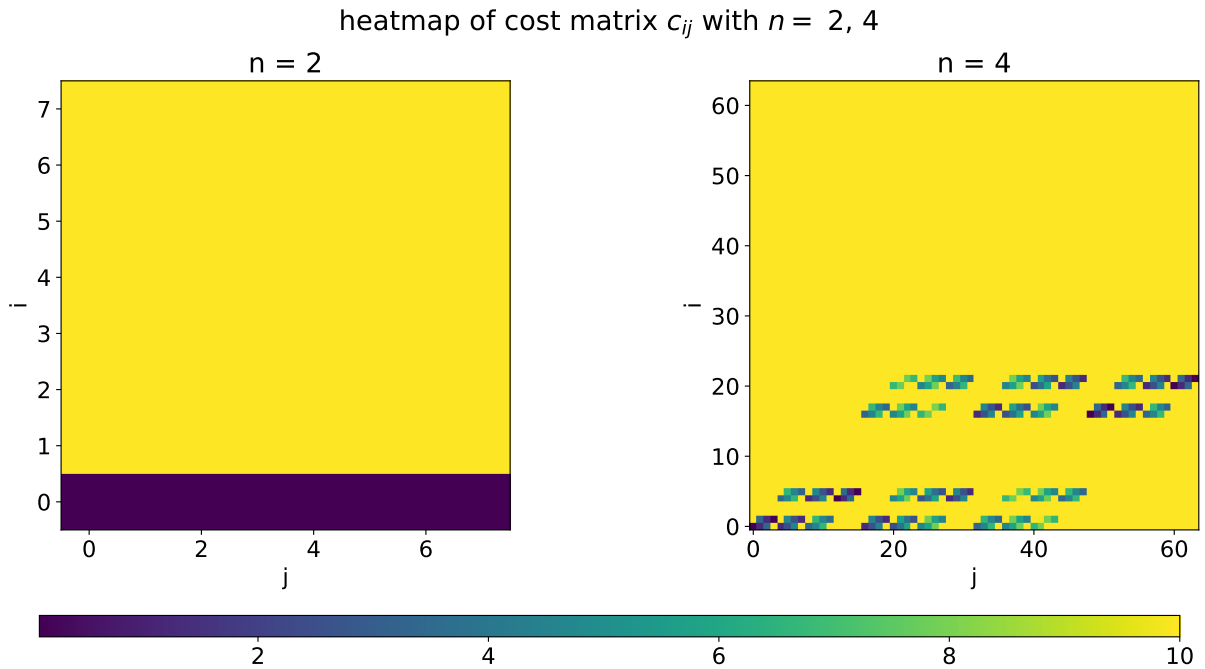


Figure 5.2: Heat map of the cost matrix for non-uniform linear motion from $[0, 50]^3$, for $n = 2$ (left) and $n = 4$ (right), as retrieved from Algorithm 2. The $n = 2$ case is not scaled as described in Section 5.1.

Note that, in general, the $n = 2$ case is not the averaged $n = 4$ case, due to the way the boxes are numbered, as box 0 does not split into box 0, 1, 2 and 3.

The same can be done for non-uniform motion. Let all the particles, again, go from $[0, 50]^3$, but add a non-linearity in the velocity. Now, the particles have x -, y - and z -velocities, which are not necessarily the same. Thus, particles can end up anywhere in $[0, 100]^3$. Again, it is expected that $\hat{\pi}_{ij}$ is high for a low i (going from $[0, 50]^3$), and $\hat{\pi}_{ij}$ can be high for any j , since the particles can end up anywhere in $[0, 100]^3$. Hence, we can expect that c_{ij} is low for low i . Furthermore, more low-cost entries of c are expected than in the uniform case. After all, all the particles did not move the same amount in x , so they might have gone to different boxes.

As we can see in Figure 5.2, in the case $n = 2$ we have indeed a very low cost moving away from box 0 (where all particles started), and a high cost everywhere else, which is what was expected. Note that the $n = 2$ case in Figure 5.2 is, again, not scaled as described in Section 5.1, as this scaling would result in a uniform heat map.

For $n = 4$ in Figure 5.2, we can see more structure, but notably still only low cost for low values of i . The structure that can be seen is indeed a structure we would expect; there is some motion from the boxes in $[0, 50]^3$ to most other boxes.

Thus, we can see that there are clear differences in the shape of the cost matrix with particle movements of constant and non-uniform velocity, which can be useful to determine structures in the cost matrix from the simulations.

5.1.2 Voids

Forms of motion that are more common in the evolution of the Universe, are expansion and collapse (Planelles et al., 2014). Let us consider (uniform and non-uniform) expansion, which tends to happen in voids. Again, a box size of 100 (arbitrary units) is used. All data points start in a cube centered at $(50, 50, 50)$ with sides of 50, and particles can move towards the whole box.

We start by considering uniform expansion along three (spatial) dimensions. A scale factor can be calculated by using the initial and final radius, while scaling the points accordingly. Because we are considering particle to expand from a certain point, we can expect a symmetric cost matrix. Note that the axis of symmetry does not necessarily have to be the line $y = x$ (from $(0, 0)$ to (n^3, n^3)), as the center of expansion is not necessarily in the center of a box, or in box $n^3/2$. Furthermore, for the $n = 2$ case, no motion between boxes can be expected, as there is no side-to-side motion. This is indeed what we can see in Figure 5.3; we see a diagonal cost matrix. Note that the one square that has the same colour as the background, c_{55} , is due to the scaling as described in Section 5.1 and this is not inherent to the system.

The case for $n = 4$, which can also be seen in Figure 5.3, looks a bit more interesting. We can see that the cost seems to be almost symmetric in an almost diagonal line, which is what is to be expected for expansion. If we count the boxes, we can see that boxes 21, 22, 25, 26, 37, 38, 41 and 42 are indeed in the center of the (full) box. Movement from those boxes outwards, which is where we can see a lower cost in Figure 5.3, is indeed what we would expect.

Now, let us look at non-uniform expansion, again along three spatial dimensions. Here,

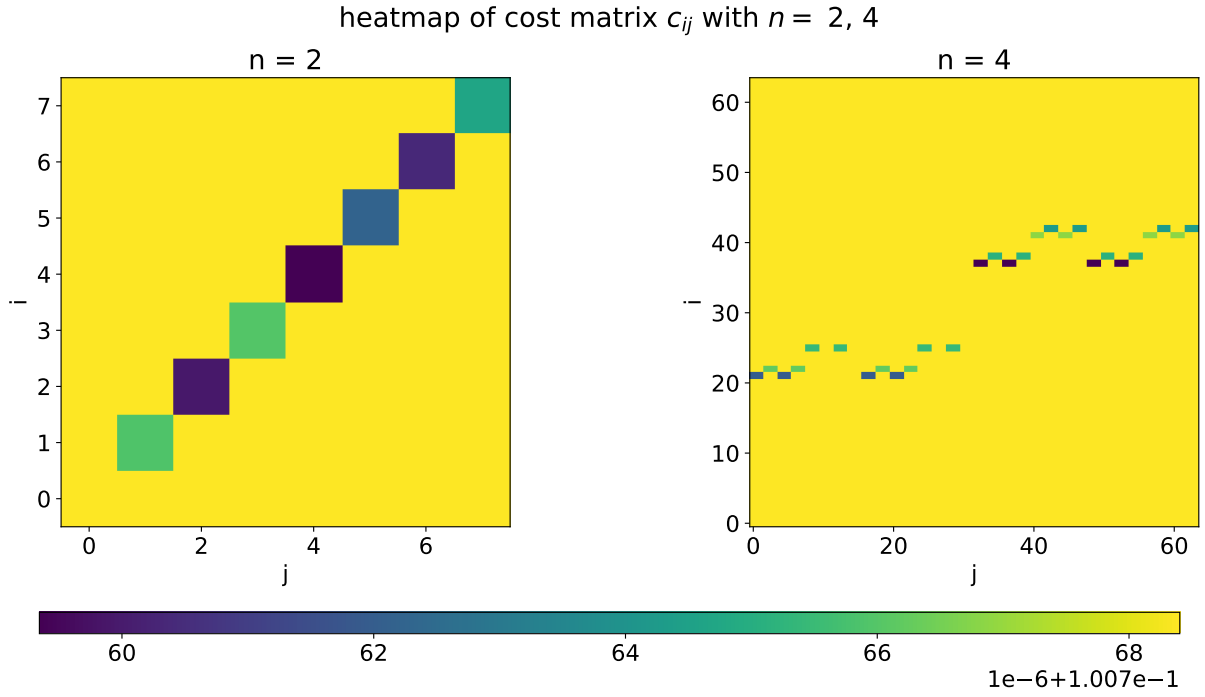


Figure 5.3: Heat map of the cost matrix for uniform expansion, for $n = 2$ (left) and $n = 4$ (right), as retrieved from Algorithm 2.

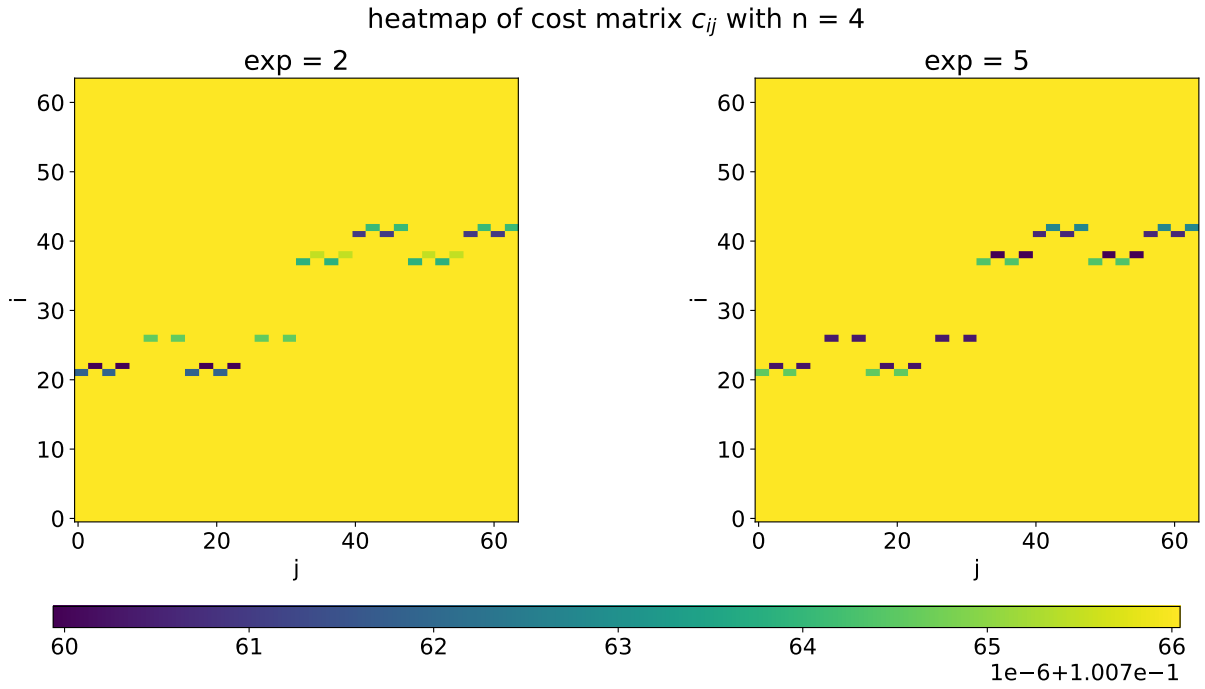


Figure 5.4: Heat map of the cost matrix for non-uniform expansion, for both $\text{exp} = 2$ (left) and $\text{exp} = 5$ (right), for $n = 4$, as retrieved from Algorithm 2.

one should choose how to simulate the non-uniformity. Physically, it makes the most sense to have accelerating expansion from a certain central point, in our case taken as the center point of our begin cube of points, which is the middle of the full box. The points will have an exponential velocity, depending on the distance from the center. This non-uniformity is implemented by using a parameter `exp`. When points are expanding from the initial sphere to a final, larger sphere, `exp` controls the distribution of the radial displacement of each particle. Specifically, the initial distance of each particle from the center of expansion is calculated, and then normalized. This gives a set of normalized distances, `normalized_distances`. These normalized distances then undergo a transformation where each element is raised to the power of `exp`, `scaled_normalized_distances = normalized_distances ** exp`.

Note that if `exp = 1`, the expansion is uniform. For `exp > 1`, there is proportionally smaller displacement for particles closer to the center of expansion, than for particles that started far from the center of expansion, leading to a relatively dense distribution of particles near the center, and a relative thin distribution of particles further from the center of expansion.

On the contrary, if $0 < \text{exp} < 1$, points that are initially closer to the center of expansion will experience a proportionally larger outward displacement compared to particles that were further away initially.

First, `exp` is chosen to be 2. For the $n = 2$ case a fully diagonal cost matrix is again expected, as we still do not expect particles to move out of their boxes. Furthermore, the results for higher n are very similar to those of the uniform expansion cases. The case $n = 4$ can be seen in Figure 5.4. It is clear that the only difference to the $n = 4$ case for uniform expansion is the scaling. Note that the "missing" squares in row 37 or row 41 in Figure 5.4 are not inherent to the system, but rather a result of the scaling that was described at the beginning of this chapter.

It is clear that the differences between uniform and non-uniform expansion are more subtle than that of uniform and non-uniform linear motion; the shapes of the cost matrix are the same. The differences are in the scaling, rather than the shape of the cost matrix. To see if there are differences visible for larger values of `exp`, another exponent was tested, `exp = 5`, as can also be seen in Figure 5.4, again for the $n = 4$ case. We can indeed see that there is a different scaling for `exp = 5`.

Note that the shapes of the cost matrices for `exp = 5` are still similar to the case of `exp = 2`, but the scaling indeed looks a bit different again.

5.1.3 Clusters

A third kind of motion to look at is collapse along three (spatial) dimensions, which tends to happen in clusters. For uniform and non-uniform collapse, we can do the same thing as we did for uniform and non-uniform expansion, but with a scale factor < 1 . Here, we start with points in the whole box, of length 100. The particles will end in a cube with sides of length 50, centered at (50, 50, 50). Due to the way the data points are generated, as with the expansion cases, a fully diagonal cost matrix is expected for the $n = 2$ cases, so we will not go into this case any further.

For uniform collapse, we can see for the case $n = 4$, in Figure 5.5, what looks like a

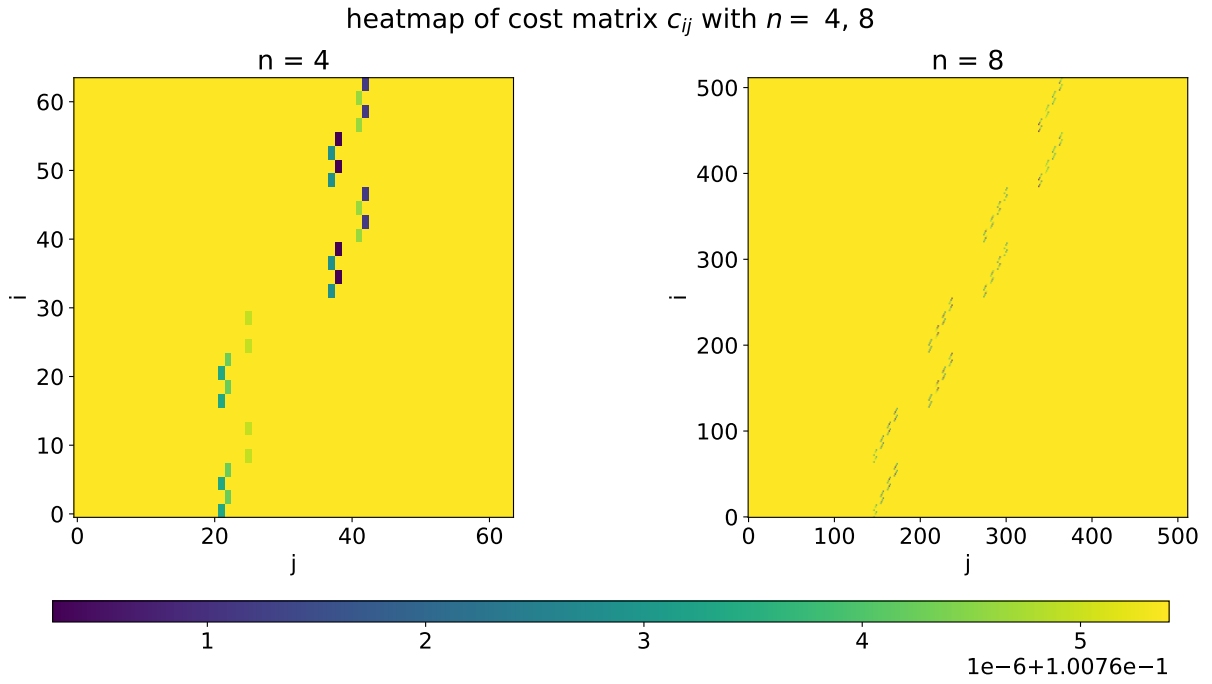


Figure 5.5: Heat map of the cost matrix for uniform collapse, for $n = 4$ (left) and $n = 8$ (right), as retrieved from Algorithm 2.

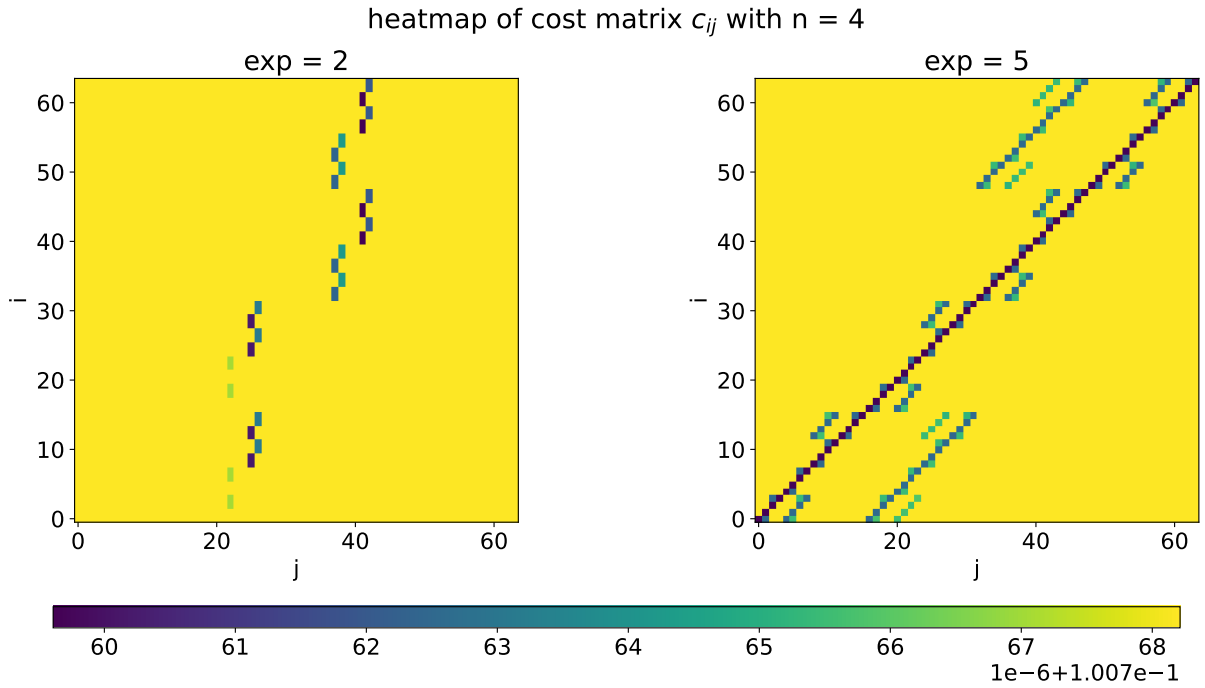


Figure 5.6: Heat map of the cost matrix for non-uniform collapse, for both $\text{exp} = 2$ (left) and $\text{exp} = 5$ (right), for $n = 4$, as retrieved from Algorithm 2.

90 degree rotation of the cost matrix with respect to the uniform expansion case, in Figure 5.3. This is to be expected, as if we time-invert the expansion, we get a collapse process. Thus, you would expect $\hat{\pi}_{ij}^c = \hat{\pi}_{ji}^e$, with $\hat{\pi}^c$ the transportation matrix of collapse, and $\hat{\pi}^e$ the transportation matrix of expansion. Hence, we would also expect $c_{ij}^c \approx c_{ji}^e$, again with c^c the cost matrix of collapse, and c^e the cost matrix of expansion. This rotation of the expansion case is indeed what we can see in Figure 5.5.

For the $n = 8$ case, also in Figure 5.5, we can see that the structure is quite similar to the $n = 4$ case. Again, the values of where the cost is lower can be explained by counting out the boxes and seeing where we would physically expect particles to go.

If we look at non-uniform collapse, we expect for lower `exp` structures that are quite similar to those of non-uniform expansion. In Figure 5.6 we can see the $n = 4$ case for `exp` = 2, which looks like a 90 degree rotation of Figure 5.4, the non-uniform expansion case for $n = 4$ and `exp` = 2.

However, when we increase `exp`, we can see different structures emerging. In Figure 5.6, for $n = 4$ and `exp` = 5, a wide, more diagonal structure can be seen, clearly distinguishable from the earlier `exp` = 2 case. Note that in this case, the cost matrix is clearly not a rotation of the expansion cost matrix in Figure 5.4. This discrepancy is probably due to the fact that the process of non-uniform expansion and collapse are not time-reversible in the same way as uniform expansion and collapse. The main reason for this discrepancy could be the asymmetry that is introduced by the non-uniformity.

5.1.4 Walls

Another common type of motion in cosmology is a combination of expansion and collapse. One example of combination of expansion and collapse is walls (or sheets). Walls are flattened, sheet-like structures that enclose the voids, and are associated with collapse along one dimension (perpendicular to the wall), and expansion along the other two dimensions (within the plane of the wall) (Bos, 2016). Simulations were made of these walls, again with a box size of 100 (arbitrary units), and points started in a cube centered at (50, 50, 50) with sides of length 50. Different values for `exp` for the expansion and collapse can be taken. We chose to consider the case where `exp` := `exp`_{collapse} = `exp`_{expansion}. The cases `exp` = 2 and `exp` = 5 were studied, as can be seen in Figure 5.7 for $n = 4$.

In Figure 5.7, it can be seen that the values of the cost matrix that are not M_c are around the same places as in the expansion and collapse cases. However, a different structure than in the pure collapse and expansion cases can be seen, with less values not equal to M_c . Because the initial condition for $n = 4$ was a cube that was only in eight boxes to begin, box 21, 22, 25, 26, 37, 38, 41 and 42, we would only expect a non- M_c cost in these boxes on the i -axis. This is indeed what we can see in both figures in Figure 5.7. Then, with collapse along one dimensions, one would not expect much movement outward in the direction of this dimension, adding diagonal values to the cost matrix. However, from expansion along the other two dimensions, one would expect the particles to move towards other boxes, corresponding to the off-diagonal values.

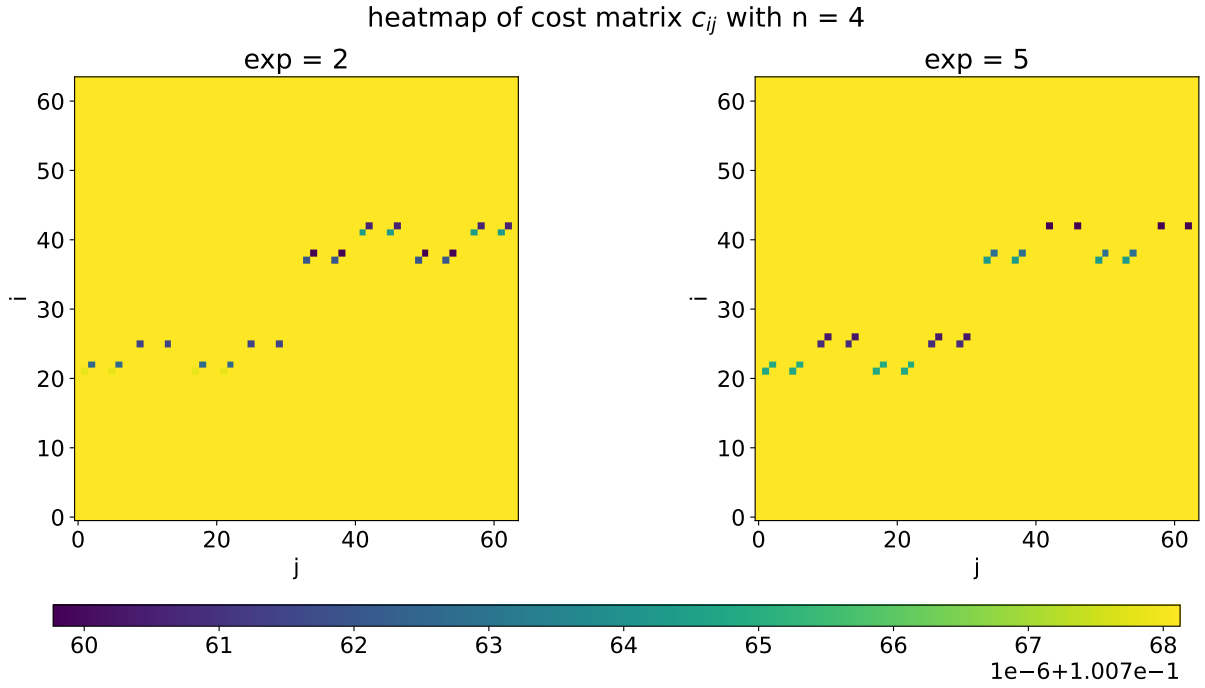


Figure 5.7: Heat map of the cost matrix for walls, for both $\text{exp} = 2$ (left) and $\text{exp} = 5$ (right), for $n = 4$, as retrieved from Algorithm 2.

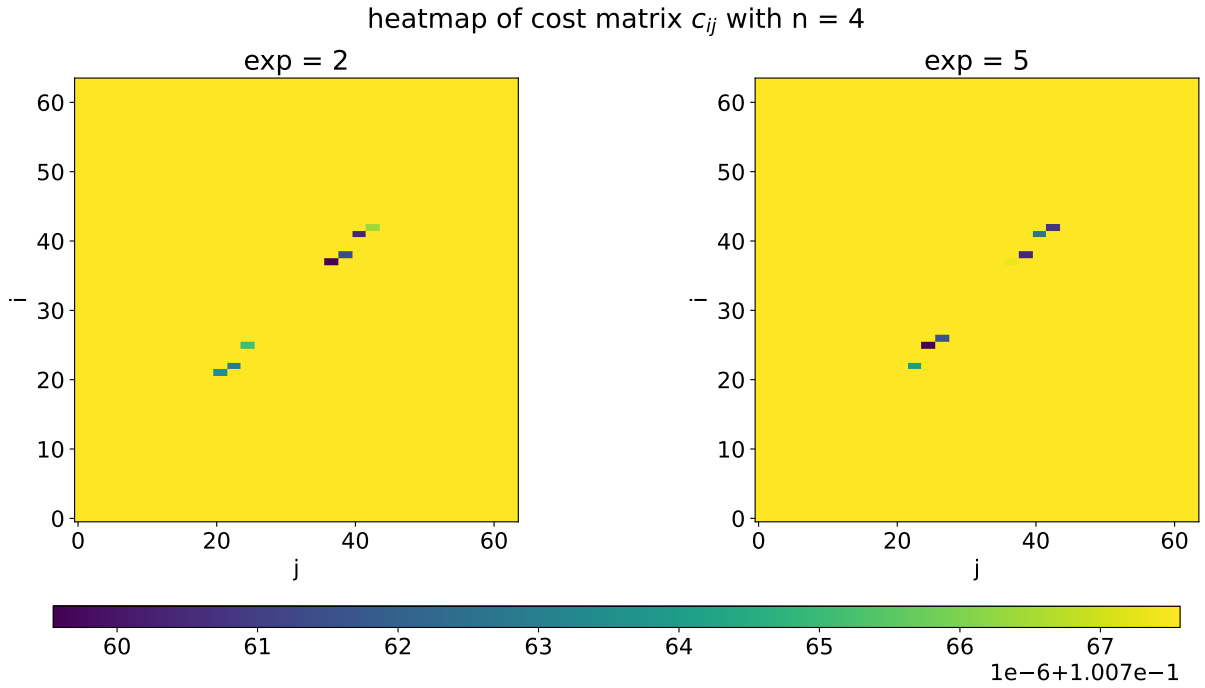


Figure 5.8: Heat map of the cost matrix for filaments, for both $\text{exp} = 2$ (left) and $\text{exp} = 5$ (right), for $n = 4$, as retrieved from Algorithm 2.

5.1.5 Filaments

As a last reference case, we will look at another mixed expansion-collapse case, filaments. Filaments are associated with collapse along two dimensions (perpendicular to the filament's axis), and expansion/flow of matter along the other dimension (Bos, 2016). We will model the latter as expansion. These simulations were made again with a box size of 100 (arbitrary units), and points started in a cube centered at (50, 50, 50) with sides of length 50. Different values of exp for the expansion and collapse can be taken. We chose, as for the walls, to consider the case where $\text{exp} := \text{exp}_{\text{collapse}} = \text{exp}_{\text{expansion}}$. Again, the cases $\text{exp} = 2$ and $\text{exp} = 5$ were studied, as can be seen in Figure 5.8 for $n = 4$.

We can see that for $n = 4$, there are very few cases where the cost matrix is not equal to M_c , all of which are approximately on the diagonal. This is probably due to the case that we started with a cube that was only in eight boxes to begin with, box 21, 22, 25, 26, 37, 38, 41 and 42. Therefore, on the i -axis, we would only expect a non- M_c cost here, which is indeed what we can see in both figures in Figure 5.8. Then, with collapse along two dimensions, one would not expect much movement outward. Hence, the particles are expected to stay in, or close to, their own boxes in the $n = 4$ case.

5.2 FLAMINGO Cost Matrix

Using Algorithm 2, the cost matrix of the FLAMINGO data, as described in Chapter 3, can be found. With the reference cases described in Section 5.1, we hope to interpret this cost matrix. First, we will comment on the expected cost matrix, whereafter the cost matrix as recovered from Algorithm 2 will be discussed.

5.2.1 Expected Cost Matrix

As described before in Chapter 3, the FLAMINGO simulations are hydrodynamical cosmological simulations, with the aim to understand the large-scale structure formation of the universe. Hence, knowledge about the expected large-scale structures can be used to comment on the expected cost matrix.

Structure Formation in the Cosmic Web

In large-scale structure formation, one would expect filaments, voids, sheets/walls and clusters to form, arising from the gravitational collapse of a relatively uniform initial density distribution in the early Universe. These different structures have different motions associated with them (Bos, 2016). Voids are vast, under-dense regions, largely without many galaxies or matter. They are characterized by expansion along all three dimensions, effectively pulling matter out of the void towards the surrounding denser structures. Walls (or sheets) are flattened, sheet-like structures that enclose the voids. The walls are denser than voids, but less dense than filaments or clusters. As mentioned before, walls are associated with collapse along one dimension (perpendicular to the wall), and expansion along the other two dimensions (within the plane of the wall). Hence, matter is flowing inwards towards the central plane of the wall, while stretching out within that plane as it gets pulled towards the denser filaments and clusters. Filaments are elongated, thread-like structures. Filaments primarily have collapse along two dimensions (perpendicular to the filament's axis), and expansion/flow of matter along the other dimension. Lastly, clusters

are the densest and most massive regions in the Cosmic Web. They are gravitationally bound, and therefore have (gravitational) collapse in all three dimensions. As we expect all of these cases to occur in our data, we expect the cost matrix to be some combination of these cases, and therefore a combination of our reference cases.

Geodesics

On a single-particle level, the particles are expected to take the locally shortest path between their initial and final position. Therefore, the particles are expected to move along the geodesics as recovered from General Relativity. Hence, the cost matrix is expected to comply with the laws of General Relativity and the Λ CDM paradigm, as the FLAMINGO simulations are based on these laws (Schaye et al., 2023). Thus, if one were to have a very high resolution, in our case computational resolution, the problem of the search for a cost matrix could be stated in a way to be a test of General Relativity. If one were to observe particle motion in larger-scale structures in the real universe and find a cost matrix from those movements, one could check the observed cost matrix against the theoretical cost matrices from the simulations. With this comparison, one could possibly infer the cosmological parameters. However, this goes beyond the scope of this thesis, so we will not discuss it any further.

5.2.2 Recovered Cost Matrix

With the knowledge of what we expect for the cost matrix in mind, Algorithm 2 can be implemented the same way as for the reference cases in Section 5.1 to find the cost matrix of the FLAMINGO data. The results can be seen in Figure 5.9, scaled in the same way as described in Section 5.1. For the $n = 4$ case, in Figure 5.9a, we can already see that the structure of the cost matrix is more complicated than the previous reference cases, which is what is to be expected. It is expected that there is quite a lot of local collapse and expansion going on, in contrast to the only one scale of expansion and collapse in the reference cases. However, this makes it difficult to interpret the cost matrix on first sight.

For $n = 8$, in Figure 5.9b, the cost matrix looks quite similar to the $n = 4$ case, still with the diagonal stripes. However, it is also clear that this is quite far from any reference case in Section 5.1.

As touched on before, it is expected that there is a lot of different kinds of motion going on in the simulation, and it is difficult to visually distinguish the different motions from these cost matrices. Note, however, that both cost matrices in Figure 5.9 look quite symmetric, which implies isotropy and is what is to be expected from the cosmological principle.

5.2.3 Markov Chain Monte Carlo Simulation

One way to interpret these results is through the assumption that the FLAMINGO results are a linear combination of multiple different reference cases.

To test this assumption, simulations of more reference cases were done, without the simulation necessarily filling the whole box. For better computational efficiency, 500 thousand particles were simulated instead of 1 million as in Section 5.1. Particles were simulated to expand from a cube of length 50 (arbitrary units) to a cube of length 100, or to collapse from a cube of length 100 to a cube of length 50. The box size was chosen to be 400.

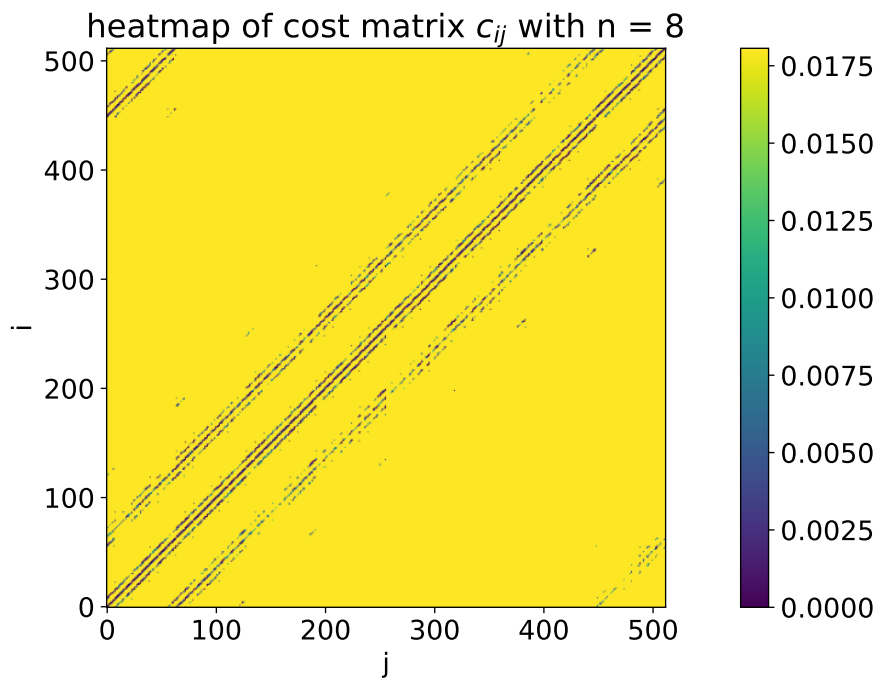
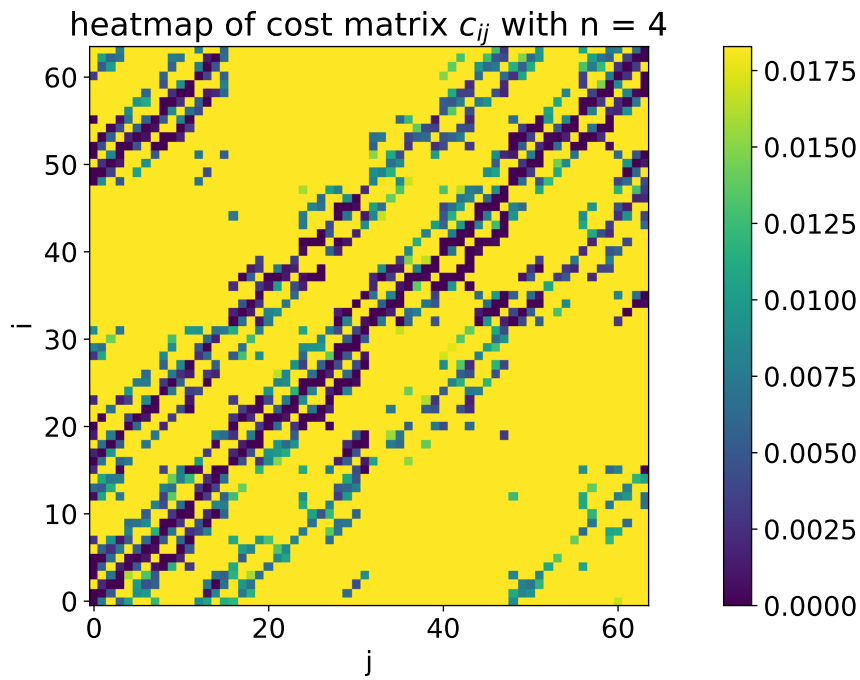


Figure 5.9: Heat map of the cost matrix for the FLAMINGO data, for $n = 4$ and $n = 8$, as retrieved from Algorithm 2.

Multiple centers for the expansion were chosen: (x, y, z) with $x, y, z \in \{100, 200, 300\}$, as well as multiple values of exp , $\text{exp} \in \{0.4, 1, 2, 5\}$. With this, 27 reference cases for each type of motion were made, all with 4 exp values, and both for expansion and collapse. This resulted in 216 reference cases. However, it is expected that this is not enough, as there is only expansion and collapse possible from a limited number of points in the box.

As a proof of concept to test the hypothesis that the FLAMINGO cost matrix is a linear combination of the reference cases, a Markov Chains Monte Carlo (MCMC) simulation was used. For MCMC, one needs to consider Bayes' formula ([Grimmett and Welsh, 2014](#)):

$$\mathbb{P}(A|B) \propto \mathbb{P}(B|A)\mathbb{P}(A), \quad (5.1)$$

with $\mathbb{P}(A|B)$ the posterior probability, $\mathbb{P}(B|A)$ the likelihood and $\mathbb{P}(A)$ the prior. Here, A can be seen as the model, and B as the data. A Gaussian prior was used,

$$\mathbb{P}(A) = \prod_i e^{-(\theta_i - \hat{\theta}_i)^2 / 2\sigma_i^2}, \quad (5.2)$$

with θ a vector of model parameters, $\hat{\theta}_i$ the initial guess for parameter θ_i , and σ_i the width of the prior. Note that σ_i should be large enough to allow the model to move away from the initial guess $\hat{\theta}$, but not so large that it jumps into regions deemed unrealistic and therefore having a higher computational cost than necessary. To ensure numerical stability, a simple bound that returns negative infinity if any of the input elements were bigger than some limit was added to this Gaussian prior, where we used a limit value of 1000. A similar bound on the parameter values was implemented to ensure all parameters are non-negative. For the likelihood, a Gaussian was also used. For all σ_i , a value of 0.5 was used.

The Python library `emcee` is used for the MCMC simulation ([Foreman-Mackey et al., 2013](#)). The simulation was run by 1000 walkers and 5000 steps for each walker. We will get some randomly generated, uniformly distributed starting positions, and set up and run a sampler afterwards. Then, we will need to look at the best values for the fit, and marginalize the data, using

$$P(a|B) = \int P(a, b|B) db, \quad (5.3)$$

where a and b are parameters of the fit. The median of the distribution will be calculated as the best value, and the 16th and 84th percentiles as the uncertainty range.

For the MCMC, we used cost matrices with $n = 8$, so 8^3 boxes. Cost matrices with $n = 8$ for the reference cases are discussed in [Appendix C](#). We can see the evolution of the walkers for the first few parameters in [Figure 5.10](#).

We can see that the chains are not converging well. This aligns with our expectation that the 216 described reference cases are not enough. The expectation is that more reference cases will solve this issue, but we cannot rule out the possibility that the FLAMINGO data is not a linear combination of the described reference cases or our initial guesses of parameters were too far off.

The cost matrix from these MCMC results can be seen in [Figure 5.11](#). As described before, the median value is used as best-fit parameter. It can be seen that the values of this cost matrix are indeed far off from the FLAMINGO data, in [Figure 5.9](#).

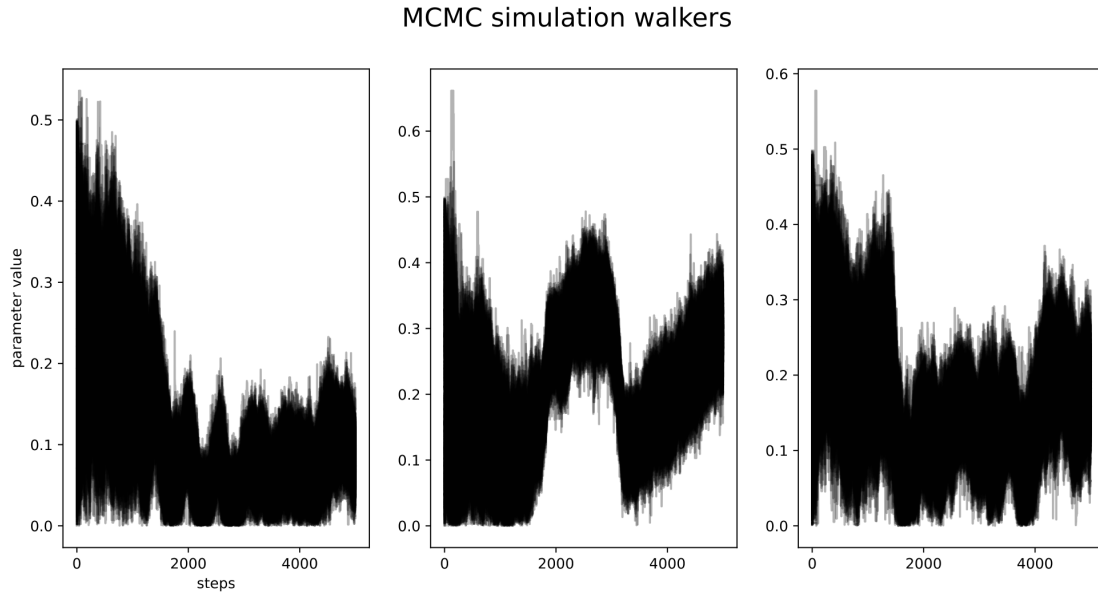


Figure 5.10: MCMC walkers for the first few parameters; expansion from $(100, 100, 100)$ (left), expansion from $(100, 100, 200)$ (middle) and expansion from $(100, 100, 300)$ (right).

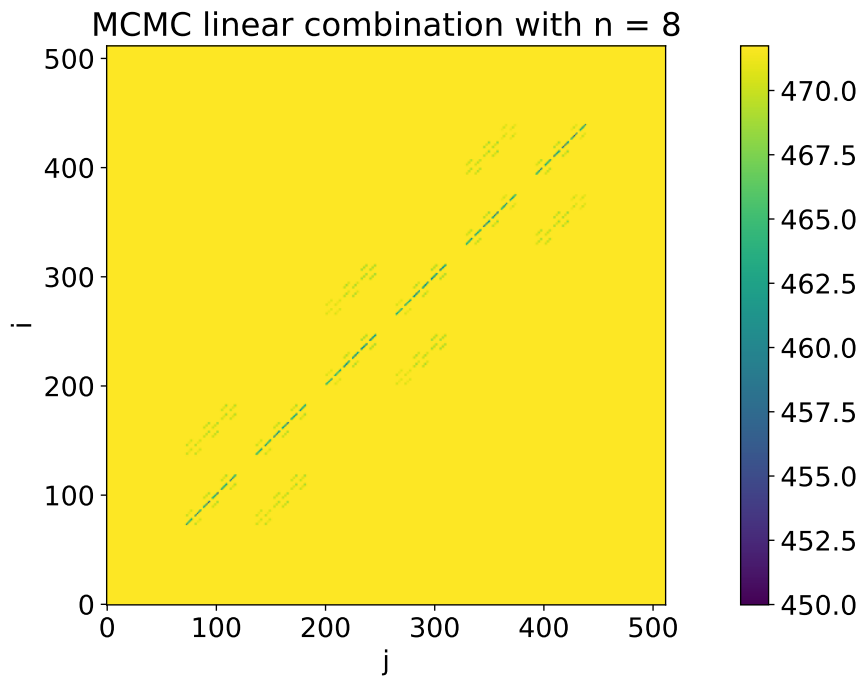


Figure 5.11: Cost matrix from the linear combination of the MCMC, with $n = 8$.

To better show how far off this cost matrix is, the differences between the FLAMINGO cost matrix and the cost matrix from the linear combination of the MCMC can be seen in Figure 5.12. It is clear that the MCMC fit is not a good fit and does not describe the data well.

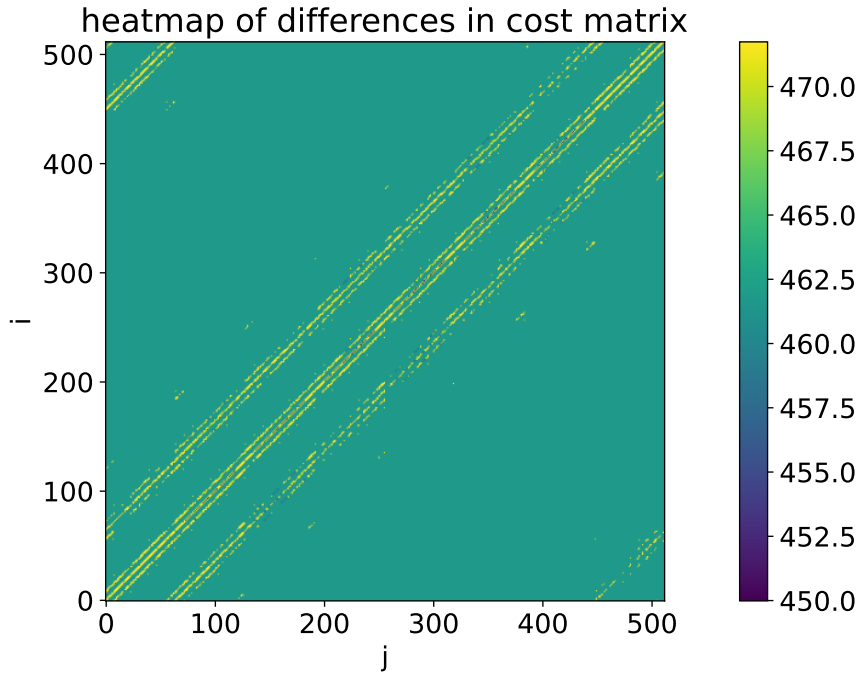


Figure 5.12: Differences between the FLAMINGO cost matrix and the cost matrix from the linear combination of the MCMC, for $n = 8$.

6. *Outlook*

The research presented in this thesis aimed to characterize the evolution of the Universe through the lens of inverse Optimal Transport (OT). The FLAMINGO simulations were chosen as suitable simulations to work with, and the data was divided into smaller boxes for computational efficiency. The aim was to find a cost matrix that describes the motion of the particles within the simulations. In Chapter 4, an algorithm was developed for applying inverse OT, and comments were made on the equivalence of different solutions, as in general inverse OT problems can be ill-posed. This algorithm was inspired by [Ma et al. \(2020\)](#), with their proofs being discretized as can be seen in Appendix A. The implementation of this algorithm was chosen in such a way to ensure numerical stability and computational efficiency.

Chapter 5 implements the algorithm, and finds the cost matrix for certain reference cases before embarking on the quest to find the cost matrix for the FLAMINGO data. We looked at linear motion, expansion, collapse and combinations of expansion and collapse. These motions are expected in the universe in voids, clusters, walls and filaments, and therefore we would expect these reference cases to say something about the FLAMINGO cost matrix. All cost matrices were seen to be different for different parameters. The cost matrices between different kinds of motion, for example between expansion and collapse, were seen to have different structures. The cost matrix between the same kind of motion with different parameters often had a different scaling (expansion) or different structure (collapse). This can help in distinguishing the different reference cases in the FLAMINGO cost matrix.

The cost matrix from the FLAMINGO data looks quite symmetric, which is to be expected from the assumption of an isotropic Universe. The hypothesis is that the FLAMINGO cost matrix is a linear combination of multiple reference cases. As a proof of concept to further analyze the cost matrix, a Markov Chain Monte Carlo simulation was made to test this hypothesis with 216 reference cases. However, the Markov chain did not converge, making the resulting cost matrix was quite different from the FLAMINGO cost matrix. The expectation is that more reference cases will solve this issue, but we cannot rule out the possibility that the FLAMINGO data is not a linear combination of the described reference cases or our initial guesses of parameters were too far off.

6.1 *Future Directions*

First and foremost, a very limited amount of reference cases is considered in this thesis, with the only possibility of expansion and collapse at certain points in the box. More than likely, this is not what is actually going on. For a more representative view, one should

have more reference cases, on different scales, or work with another way of interpreting the cost matrix of the data.

Furthermore, a reoccurring theme in this thesis was the computational limits. Hence, future research is suggested to look into the computational efficiency of assigning the data into boxes, or finding the transportation matrix $\hat{\pi}$ through an alternative route, achieving better resolution possibly with more particles in larger simulations, possibly recovering the geodesics. Furthermore, future research could focus on different regularization functions $R(c)$, to see if any significantly different, non-equivalent solutions can be found.

The ultimate goal of the framework of inverse OT applied to cosmology is to apply it to larger simulations than we did in this thesis. Then, one could compare the cost matrix of different simulations with different parameters, to eventually compare this theoretical cost matrix to real-world data. Hence, if we compare the real-world data to different simulations with different parameters, the presented framework can be used as a test for General Relativity. However, retrieving any real-world data can be difficult, as to have similar data that was used in the simulations in this thesis, the Cosmic Web must be observed in detail and over a large enough timescale. While clusters of galaxies have been extensively studied and observed, far less is known about filaments, walls and voids (Bos, 2016). Filaments, walls and voids are by nature hard to observe due to their low density and surface brightness. Hence, observing these objects is difficult, which makes it even more difficult to find the dynamics of these objects. One alternative to trying to observe the full dynamics of the Cosmic Web, might be finding the cost matrix of closer-by, rapidly evolving stellar systems, such as the stars around Sgr A*, the black hole in the galactic center. These stars have been studied extensively, both by numerical simulations and empirical observations, and evolve quite rapidly (Gillessen et al., 2009; Zwart et al., 2023). This makes this system a possible candidate for inverse OT.

6.2 Acknowledgments

First of all, I would like to thank my Matthieu for introducing me to this project, and his support and idead throughout. I would also like to thank Elena for her interesting suggestions and questions. Besides, I would like to thank Dirk for last-minute stepping up to read through my thesis in the last few weeks of writing. Also, many thanks to Rob McGibbon for helping me with the data reduction when my own Python skills were lacking. Lastly, I want to thank my dad for checking the language of this report.

For making the figures, GitHub Copilot was used to generate the first basic layout of the code.

Bibliography

- Beck, A. (2015). On the convergence of alternating minimization for convex programming with applications to iteratively reweighted least squares and decomposition schemes. *SIAM Journal on Optimization*, 25(1):185–209.
- Benamou, J.-D. and Brenier, Y. (1998). The optimal time-continuous mass transport problem and its augmented lagrangian numerical resolution. *Numer. Math.*, 84:375–393.
- Benamou, J.-D. and Brenier, Y. (2000). A computational fluid mechanics solution to the monge-kantorovich mass transfer problem. *Numerische Mathematik*, 84:375–393.
- Borrow, J. and Borrisov, A. (2020). swiftsimio: A python library for reading swift data. *Journal of Open Source Software*, 5(52):2430.
- Bos, E. G. P. (2016). *Clusters, voids and reconstructions of the cosmic web*. Phd thesis, University of Groningen.
- Chiu, W.-T., Wang, P., and Shafto, P. (2022). Discrete probabilistic inverse optimal transport.
- Crain, R. A., Schaye, J., Bower, R. G., Furlong, M., Schaller, M., Theuns, T., Dalla Vecchia, C., Frenk, C. S., McCarthy, I. G., Helly, J. C., Jenkins, A., Rosas-Guevara, Y. M., White, S. D. M., and Trayford, J. W. (2015). The eagle simulations of galaxy formation: calibration of subgrid physics and model variations. *Monthly Notices of the Royal Astronomical Society*, 450(2):1937–1961.
- Cuturi, M. (2013). Sinkhorn distances: Lightspeed computation of optimal transportation distances.
- Durrer, R. (2015). The cosmic microwave background: the history of its experimental investigation and its significance for cosmology. *Classical and Quantum Gravity*, 32(12):124007.
- Foreman-Mackey, D., Hogg, D. W., Lang, D., and Goodman, J. (2013). emcee: The mcmc hammer. *Publications of the Astronomical Society of the Pacific*, 125(925):306–312.
- Gillessen, S., Eisenhauer, F., Trippe, S., Alexander, T., Genzel, R., Martins, F., and Ott, T. (2009). Monitoring stellar orbits around the massive black hole in the galactic center. *The Astrophysical Journal*, 692(2):1075–1109.
- Grimmett, G. and Welsh, D. (2014). *Probability an Introduction*. Oxford University Press, 2 edition.

- Korte, B. and Vygen, J. (2018). *Combinatorial Optimization*. Algorithms and Combinatorics. Springer Berlin, Heidelberg, 6 edition.
- Kugel, R., Schaye, J., Schaller, M., Helly, J. C., Braspenning, J., Elbers, W., Frenk, C. S., McCarthy, I. G., Kwan, J., Salcido, J., van Daalen, M. P., Vandenbroucke, B., Bahé, Y. M., Borrow, J., Chaikin, E., Huško, F., Jenkins, A., Lacey, C. G., Nobels, F. S. J., and Vernon, I. (2023). Flamingo: Calibrating large cosmological hydrodynamical simulations with machine learning.
- L. R. Ford, J. and Fulkerson, D. R. (1962). *Flows in Networks*. Princeton Landmarks in Mathematics and Physics. Princeton University Press.
- Li, R., Ye, X., Zhou, H., and Zha, H. (2019). Learning to match via inverse optimal transport. *Journal of Machine Learning Research*, 20(80):1–37.
- Ma, S., Sun, H., Ye, X., Zha, H., and Zhou, H. (2020). Learning cost functions for optimal transport.
- Monge, G. (1781). Mémoire sur la théorie des déblais et des remblais. *Histoire de l'Académie Royale des Science de Paris*, pages 666–704.
- Nemirovski, A. and Rothblum, U. (1999). On complexity of matrix scaling. *Linear Algebra and its Applications*, 302:435–460.
- Nikakhtar, F., Sheth, R. K., Padmanabhan, N., Lévy, B., and Mohayaee, R. (2024). Displacement field analysis via optimal transport: Multi-tracer approach to cosmological reconstruction.
- Peyré, G. and Cuturi, M. (2019). Computational optimal transport.
- Planelles, S., Schleicher, D. R. G., and Bykov, A. M. (2014). Large-scale structure formation: From the first non-linear objects to massive galaxy clusters. *Space Science Reviews*, 188(1-4):93–139.
- Schaye, J., Crain, R. A., Bower, R. G., Furlong, M., Schaller, M., Theuns, T., Dalla Vecchia, C., Frenk, C. S., McCarthy, I. G., Helly, J. C., Jenkins, A., Rosas-Guevara, Y. M., White, S. D. M., Baes, M., Booth, C. M., Camps, P., Navarro, J. F., Qu, Y., Rahmati, A., Sawala, T., Thomas, P. A., and Trayford, J. (2014). The eagle project: simulating the evolution and assembly of galaxies and their environments. *Monthly Notices of the Royal Astronomical Society*, 446(1):521–554.
- Schaye, J., Kugel, R., Schaller, M., Helly, J. C., Braspenning, J., Elbers, W., McCarthy, I. G., van Daalen, M. P., Vandenbroucke, B., Frenk, C. S., Kwan, J., Salcido, J., Bahé, Y. M., Borrow, J., Chaikin, E., Hahn, O., Huško, F., Jenkins, A., Lacey, C. G., and Nobels, F. S. J. (2023). The flamingo project: cosmological hydrodynamical simulations for large-scale structure and galaxy cluster surveys. *Monthly Notices of the Royal Astronomical Society*, 526(4):4978–5020.
- Sinkhorn, R. (1964). A relationship between arbitrary positive matrices and doubly stochastic matrices. *Annals of Mathematical Statistics*, 35:876–879.
- Stuart, A. M. and Wolfram, M.-T. (2020). Inverse optimal transport. *SIAM Journal on Applied Mathematics*, 80(1):599–619.

- Vanderbei, R. J. (2020). *Linear Programming*. International Series in Operations Research & Management Science. Springer Cham, 5 edition.
- Villani, C. (2003). *Topics in Optimal Transportation*. Graduate studies in mathematics (American Mathematical Society, Providence (R.I.)).
- Villani, C. (2008). *Optimal transport, old and new*. Springer.
- Zel'dovich, Y. B. (1970). Fragmentation of a homogeneous medium under the action of gravitation. *Astrophysics*, 6(2):164–174.
- Zwart, S. P., Boekholt, T., and Hoggie, D. (2023). Punctuated chaos and the unpredictability of the galactic center s-star orbital evolution.

A. Proofs

A.1 Proof of Lemma 4.1

Lemma 4.1. *The dual problem of (4.3) is given by*

$$D(\alpha, \beta) := \sum_{i=1}^m \alpha_i \mu_i + \sum_{j=1}^n \beta_j \nu_j - \varepsilon \sum_{i=1}^m \sum_{j=1}^n \exp\left(\frac{\alpha_i + \beta_j - c_{ij} - \varepsilon}{\varepsilon}\right), \quad (4.4)$$

and the dual optimization problem is given by

$$\max_{\alpha \in \mathbb{R}^m, \beta \in \mathbb{R}^n} D(\alpha, \beta). \quad (4.5)$$

Proof. Consider equation (4.3) and introduce the Lagrangian multipliers $\alpha \in \mathbb{R}^m, \beta \in \mathbb{R}^n$; α and β penalize deviations from the marginal constraints. Now, at the optimum there should hold (by definition of Lagrangian multipliers)

$$g_1(i) = \sum_{j=1}^n \pi_{ij} - \mu_i = 0, \quad (A.1a)$$

$$g_2(j) = \sum_{i=1}^m \pi_{ij} - \nu_j = 0. \quad (A.1b)$$

By definition of a Lagrangian multiplier, if we want to minimize $f(z)$ subject to $g(z) = 0$, we have

$$\mathcal{L}(z, \lambda) = f(z) \pm \kappa g(z),$$

with $\mathcal{L}$ the Lagrangian, κ the Lagrangian multiplier and the $\pm$ a convention. For simplicity, let us take the expression with $+$. Note that α and β are incorporated in κ . Then, the Lagrangian is

$$\begin{aligned} \mathcal{L}(\pi, \alpha, \beta) &= \sum_{i=1}^m \sum_{j=1}^n [c_{ij} \pi_{ij} + \varepsilon \pi_{ij} \log(\pi_{ij})] \\ &\quad - \sum_{i=1}^m \alpha_i g_1(i) - \sum_{j=1}^n \beta_j g_2(j), \end{aligned}$$

with $g_1(i), g_2(j)$ as in (A.1). Expanding the terms with the multipliers gives us

$$- \sum_{i=1}^m \alpha_i \left[\sum_{j=1}^n \pi_{ij} - \mu_i \right] = - \sum_{i=1}^m \sum_{j=1}^n \alpha_i \pi_{ij} + \sum_{i=1}^m \alpha_i \mu_i.$$

Similarly,

$$-\sum_{j=1}^n \beta_j \left[\sum_{i=1}^m \pi_{ij} - \nu_j \right] = -\sum_{i=1}^m \sum_{j=1}^n \beta_j \pi_{ij} + \sum_{j=1}^n \beta_j \nu_j.$$

Thus, we see for the Lagrangian

$$\begin{aligned} \mathcal{L}(\pi, \alpha, \beta) &= \sum_{i=1}^m \sum_{j=1}^n [c_{ij} \pi_{ij} + \varepsilon \pi_{ij} \log(\pi_{ij})] \\ &\quad - \sum_{i=1}^m \sum_{j=1}^n \alpha_i \pi_{ij} + \sum_{i=1}^m \alpha_i \mu_i - \sum_{i=1}^m \sum_{j=1}^n \beta_j \pi_{ij} + \sum_{j=1}^n \beta_j \nu_j \\ &= \sum_{i=1}^m \sum_{j=1}^n [c_{ij} + \varepsilon \log(\pi_{ij}) - \alpha_i - \beta_j] \pi_{ij} + \sum_{i=1}^m \alpha_i \mu_i + \sum_{j=1}^n \beta_j \nu_j. \end{aligned} \quad (\text{A.2})$$

Now, for the dual problem, we want to minimize $\mathcal{L}$ over π for fixed α^* and β^* . Thus the minimum can be found by setting the partial derivative of $\mathcal{L}$ with respect to π to zero:

$$\frac{\partial \mathcal{L}(\pi^*, \alpha^*, \beta^*)}{\partial \pi_{ij}^*} = c_{ij}^* + \varepsilon \log(\pi_{ij}^*) - \alpha_i^* - \beta_j^* + \varepsilon = 0.$$

Then, we can see

$$-\varepsilon \log(\pi_{ij}^*) = c_{ij}^* - \alpha_i^* - \beta_j^* + \varepsilon,$$

thus

$$\pi_{ij}^* = \exp \left(\frac{\alpha_i^* + \beta_j^* - c_{ij}^* - \varepsilon}{\varepsilon} \right). \quad (\text{A.3})$$

Therefore, we can see

$$\begin{aligned} \mathcal{L}(\pi, \alpha, \beta) &- \sum_{i=1}^m \alpha_i \mu_i - \sum_{j=1}^n \beta_j \nu_j \\ &= \sum_{i=1}^m \sum_{j=1}^n [c_{ij} + \varepsilon \log(\pi_{ij}) - \alpha_i - \beta_j] \pi_{ij} \\ &= \sum_{i=1}^m \sum_{j=1}^n \left[c_{ij} + \varepsilon \log \exp \left(\frac{\alpha_i + \beta_j - c_{ij} - \varepsilon}{\varepsilon} \right) - \alpha_i - \beta_j \right] \pi_{ij} \\ &= -\varepsilon \sum_{i=1}^m \sum_{j=1}^n \exp \left(\frac{\alpha_i + \beta_j - c_{ij} - \varepsilon}{\varepsilon} \right). \end{aligned}$$

Thus, the dual problem is given by

$$D(\alpha, \beta) := \sum_{i=1}^m \alpha_i \mu_i + \sum_{j=1}^n \beta_j \nu_j - \varepsilon \sum_{i=1}^m \sum_{j=1}^n \exp \left(\frac{\alpha_i + \beta_j - c_{ij} - \varepsilon}{\varepsilon} \right),$$

and the dual optimization problem is given by

$$\max_{\alpha \in \mathbb{R}^m, \beta \in \mathbb{R}^n} D(\alpha, \beta).$$

□

A.2 Proof of Theorem 4.2

Theorem 4.2. *The optimization problem in equations (4.7a) and (4.7b) is equivalent to*

$$\min_{\alpha \in \mathbb{R}^m, \beta \in \mathbb{R}^n, c \in \mathbb{R}^{m \times n}} E(\alpha, \beta, c) + R(c), \quad (4.8)$$

with

$$E(\alpha, \beta, c) := \sum_{i,j} c_{ij} \hat{\pi}_{ij} - \sum_i \alpha_i \mu_i - \sum_j \beta_j \nu_j + \varepsilon \sum_{i,j} e^{(\alpha_i + \beta_j - c_{ij} - \varepsilon)/\varepsilon}. \quad (4.9)$$

Here, equivalent means that c^* solves (4.7) if and only if $(\alpha^{c^*}, \beta^{c^*}, c^*)$ solves (4.8) with $(\alpha^{c^*}, \beta^{c^*})$ the optimal solution for the dual problem (4.5) with cost c^* .

Proof. Recall from the proof of Lemma 4.1, equation (A.3), that the optimal transport plan is given by

$$\pi_{ij}^* = \exp \left[\frac{\alpha_i^* + \beta_j^* - c_{ij}^* - \varepsilon}{\varepsilon} \right].$$

Note that $\pi_{ij}^c > 0$ for all i, j . Now, we can see

$$\begin{aligned} \hat{c} = \text{KL}(\hat{\pi}, \pi^c) &= \sum_{i,j} \hat{\pi}_{ij} \log \left(\frac{\hat{\pi}_{ij}}{\pi_{ij}^*} \right) \\ &= \sum_{i,j} \hat{\pi}_{ij} [\log(\hat{\pi}_{ij}) - \log(\pi_{ij}^*)] \\ &= -H(\hat{\pi}) - \frac{1}{\varepsilon} \sum_{i,j} \hat{\pi}_{ij} (\alpha_i^* + \beta_j^* - c_{ij}^* - \varepsilon), \end{aligned} \quad (A.4)$$

with H as in equation (2.8), and $\hat{\pi}$ as in (4.6). Note that since μ and ν are marginals of π , we can see

$$\sum_{i,j} \alpha_i \pi_{ij} = \sum_i \alpha_i \mu_i, \quad \sum_{i,j} \beta_j \pi_{ij} = \sum_j \beta_j \nu_j. \quad (A.5)$$

Then, plugging (A.3) and (A.4) into (4.7a), we can find

$$\begin{aligned} \hat{c} &= \min_c \text{KL}(\hat{\pi}, \pi^c) + \varepsilon^{-1} R(c) \\ &= \min_c \left[\sum_{i,j} \hat{\pi}_{ij} (\log(\hat{\pi}_{ij}) - \log(\pi_{ij}^c)) \varepsilon^{-1} R(c) \right] \\ &= \min_c \left[\sum_{i,j} \hat{\pi}_{ij} \log(\hat{\pi}_{ij}) - \sum_{i,j} \hat{\pi}_{ij} \frac{\alpha_i + \beta_j - c_{ij} - \varepsilon}{\varepsilon} - \varepsilon^{-1} R(c) \right] \\ &= \min_c \left[H(\hat{\pi}) - \varepsilon^{-1} \left(\sum_{i,j} \hat{\pi}_{ij} (\alpha_i + \beta_j - c_{ij} - \varepsilon) - R(c) \right) \right] \\ &= \min_c \left[- \sum_i \alpha_i \mu_i - \sum_j \beta_j \nu_j + \sum_{i,j} c_{ij} \hat{\pi}_{ij} + R(c) \right], \end{aligned} \quad (A.6)$$

where, in the last step we used equation (A.5), and the fact that since $H(\hat{\pi})$ is independent of c and thus does not alter the minimization, and so does multiplication by some constant $\varepsilon > 0$ or addition of a scalar ($\varepsilon \sum_{i,j} \hat{\pi}_{ij} = \varepsilon$).

Now, plugging this into (4.9), we can see

$$\begin{aligned} E(\alpha^*, \beta^*, c^*) &:= \sum_{i,j} c_{ij}^* \hat{\pi}_{ij} - \sum_i \alpha_i^* \mu_i - \sum_j \beta_j^* \nu_j + \varepsilon \sum_{i,j} e^{(\alpha_i^* + \beta_j^* - c_{ij}^* - \varepsilon)/\varepsilon} \\ &= \sum_{i,j} c_{ij}^* \hat{\pi}_{ij} - \sum_i \alpha_i^* \mu_i - \sum_j \beta_j^* \nu_j + \varepsilon, \end{aligned} \quad (\text{A.7})$$

since

$$\sum_{i,j} e^{(\alpha_i^* + \beta_j^* - c_{ij}^* - \varepsilon)/\varepsilon} = \sum_{i,j} \hat{\pi}_{ij} = 1,$$

since $\hat{\pi}$ is a probability vector. Note again that adding a constant factor does not alter the minimization. However, since by assumption (α^c, β^c) is optimal for any fixed c in the dual problem (4.5), we can see

$$E(\alpha^c, \beta^c, c) = \min_{\alpha \in \mathbb{R}^m, \beta \in \mathbb{R}^n} E(\alpha, \beta, c).$$

Thus, we can see that equation 4.7b has the same set of solutions as equation (4.9). Now, if we combine this with equation (A.7), and eliminate ε , we find

$$\min_{\alpha \in \mathbb{R}^m, \beta \in \mathbb{R}^n, c \in \mathbb{R}^{m \times n}} E(\alpha, \beta, c) + R(c),$$

which is exactly (4.8), as we wanted. □

A.3 Proof of Theorem 4.4

Theorem 4.4. *The following statements are true for $E(\alpha, \beta, c)$ in (4.8):*

- (i) $E(\alpha, \beta, c)$ is jointly convex in (α, β, c) ;
- (ii) E is a constant on the equivalence class $[(\alpha, \beta, c)]$ for each $(\alpha, \beta, c) \in \mathcal{W}$;
- (iii) The functional $\tilde{E} : \tilde{\mathcal{W}} \rightarrow \mathbb{R}$ with $\tilde{E}([(\alpha, \beta, c)]) = E(\alpha, \beta, c)$ is well defined;
- (iv) If (α^*, β^*, c^*) is a solution of (4.8), then the optimal transport plan π^* in (4.7) is given by

$$\pi_{ij}^* = e^{(\alpha_i^* + \beta_j^* - c_{ij}^* - \varepsilon)/\varepsilon};$$

- (v) $[(\alpha^*, \beta^*, c^*)]$ as in (iv) is the unique minimizer of $\tilde{E}$.

Proof. (i) $E(\alpha, \beta, c)$ is jointly convex in (α, β, c) :

Let us define $\phi = (\alpha, \beta, c)$, $\psi = (\delta\alpha, \delta\beta, \delta c)$ and

$$f(v) := E(\phi + v\psi),$$

for $v \in \mathbb{R}$. Then, we can see, by filling in (4.9),

$$f(v) = \sum_{i,j} (c_{ij} + v\delta c_{ij}) \hat{\pi}_{ij} - \sum_i (\alpha_i + v\delta\alpha_i) \mu_i - \sum_j (\beta_j + v\delta\beta_j) \nu_j + \varepsilon \sum_{i,j} e^{(K_{ij} + v\Delta K_{ij} - \varepsilon)/\varepsilon},$$

with $K_{ij} := \alpha_i + \beta_j - c_{ij}$ and $\Delta K_{ij} := \delta\alpha_i + \delta\beta_j - \delta c_{ij}$. Then, we find for the derivative with respect to v :

$$\begin{aligned} f'(v) &= \sum_{i,j} \delta c_{ij} \hat{\pi}_{ij} - \sum_i \delta\alpha_i \mu_i - \sum_j \delta\beta_j \nu_j + \varepsilon \sum_{i,j} e^{(K_{ij}+v\Delta K_{ij}-\varepsilon)/\varepsilon} \left(\frac{\Delta K_{ij}}{\varepsilon} \right) \\ &= \sum_{i,j} \delta c_{ij} \hat{\pi}_{ij} - \sum_i \delta\alpha_i \mu_i - \sum_j \delta\beta_j \nu_j + \sum_{i,j} e^{(K_{ij}+v\Delta K_{ij}-\varepsilon)/\varepsilon} \Delta K_{ij}. \end{aligned}$$

And, for the second derivative:

$$\begin{aligned} f''(v) &= \sum_{i,j} e^{(K_{ij}+v\Delta K_{ij}-\varepsilon)/\varepsilon} \Delta K_{ij} \left(\frac{\Delta K_{ij}}{\varepsilon} \right) \\ &= \frac{1}{\varepsilon} \sum_{i,j} e^{(K_{ij}+v\Delta K_{ij}-\varepsilon)/\varepsilon} (\Delta K_{ij})^2 \geq 0. \end{aligned}$$

Thus, we can see that E is jointly convex in (α, β, c) .

(ii) E is constant on each equivalence class $[(\alpha, \beta, c)]$:

Suppose $(\alpha, \beta, c) \sim (\bar{\alpha}, \bar{\beta}, \bar{c})$, and denote $\delta\alpha_i := \alpha_i - \bar{\alpha}_i$ and $\delta\beta_j := \beta_j - \bar{\beta}_j$. Then, we can see

$$\bar{c}_{ij} = \bar{\alpha}_i + \bar{\beta}_j - \alpha_i - \beta_j + c_{ij} = -\delta\alpha_i - \delta\beta_j + c_{ij}.$$

Hence, we have

$$\begin{aligned} E(\bar{\alpha}, \bar{\beta}, \bar{c}) &= \sum_{i,j} \bar{c}_{ij} \hat{\pi}_{ij} - \sum_i \bar{\alpha}_i \mu_i - \sum_j \bar{\beta}_j \nu_j + \varepsilon \sum_{i,j} e^{(\bar{\alpha}_i + \bar{\beta}_j - \bar{c}_{ij} - \varepsilon)/\varepsilon} \\ &= \sum_{i,j} (-\delta\alpha_i - \delta\beta_j + c_{ij}) \hat{\pi}_{ij} - \sum_i (\alpha_i - \delta\alpha_i) \mu_i - \sum_j (\beta_j - \delta\beta_j) \nu_j \\ &\quad + \varepsilon \sum_{i,j} e^{(\alpha_i + \beta_j - c_{ij} - \varepsilon)/\varepsilon} \\ &= E(\alpha, \beta, c) - \sum_i \delta\alpha_i \left(\sum_j \hat{\pi}_{ij} \right) - \sum_j \delta\beta_j \left(\sum_i \hat{\pi}_{ij} \right) + \sum_i \delta\alpha_i \mu_i + \sum_j \delta\beta_j \nu_j \\ &= E(\alpha, \beta, c) - \sum_i \delta\alpha_i \mu_i - \sum_j \delta\beta_j \nu_j + \sum_i \delta\alpha_i \mu_i + \sum_j \delta\beta_j \nu_j \\ &= E(\alpha, \beta, c). \end{aligned}$$

where in the third step, we used the fact that μ, ν are marginals of π . Thus E is a constant on $[(\alpha, \beta, c)]$.

(iii) $\tilde{E} : \tilde{\mathcal{W}} \rightarrow \mathbb{R}$ with $\tilde{E}([(\alpha, \beta, c)]) = E(\alpha, \beta, c)$ is well-defined:

As an immediate result of (ii), we can set $\tilde{E} : \tilde{\mathcal{W}} \rightarrow \mathbb{R}$ as $\tilde{E}([\phi]) := E(\phi)$ for every $[\phi] \in \tilde{\mathcal{W}}$, then $\tilde{E}$ is well-defined.

(iv) The optimal transport plan π^* is given by $\pi^* = e^{(\alpha^* + \beta^* - c^* - \varepsilon)/\varepsilon}$:

Let (α^*, β^*, c^*) be a minimizer of (4.8). Then, (α^*, β^*) minimizes $E(\alpha, \beta, c)$ if $c = c^*$. Thus, we find

$$\begin{aligned} (\alpha^*, \beta^*) &= \operatorname{argmin}_{\alpha, \beta} E(\alpha, \beta, c^*) \\ &= \operatorname{argmin}_{\alpha, \beta} \left\{ \varepsilon \sum_{i,j} e^{(\alpha_i + \beta_j - c_{ij} - \varepsilon)/\varepsilon} - \sum_i \alpha_i \mu_i - \sum_j \beta_j \nu_j \right\}, \end{aligned}$$

where we can neglect the c -terms, as we assumed $c = c^*$ constant.

Hence, (α^*, β^*) is the optimal dual variable, thus the optimal transport plan is (from the same derivation as Lemma 4.1)

$$\pi^* = e^{(\alpha^* + \beta^* - c^* - \varepsilon)/\varepsilon}.$$

(v) $[(\alpha^*, \beta^*, c^*)]$ is the unique minimizer of $\tilde{E}$:

Let $[\phi] = [(\alpha, \beta, c)]$, $[\psi] = [(\delta\alpha, \delta\beta, \delta c)] \in \widetilde{W}$ be two distinct equivalence classes. Thus there is at least one pair $(i, j) \in X \times Y$ such that $\Delta K_{ij} := \delta\alpha_i + \delta\beta_j - \delta c_{ij} \neq 0$.

Consider the function $\tilde{f}(v) := \tilde{E}([\phi + v\psi]) = E(\phi + v\psi)$. Then, in similar fashion as in (i), its second derivative is:

$$f''(v) = \frac{1}{\varepsilon} \sum_{i=1}^m \sum_{j=1}^n e^{(K_{ij} + v\Delta K_{ij})/\varepsilon} (\Delta K_{ij})^2 > 0,$$

since $[\psi] \neq [0]$. Since we chose $[\phi]$ arbitrary, we can see $\tilde{f}$ is strictly convex at every $[\phi] \in \mathcal{W}$, thus $[\phi]$ is the unique minimizer of $\tilde{E}$ on $\mathcal{W}$.

□

A.4 Proof of Lemma 4.5

Lemma 4.5. *The inverse Optimal Transport minimization (4.8) has the same set of solutions as the following minimization:*

$$\min_{\alpha, \beta, c} \Psi(\alpha, \beta, c) := F(\alpha, \beta, c) + R(c), \quad (4.12)$$

with

$$F(\alpha, \beta, c) = -\langle \alpha, \mu \rangle - \langle \beta, \nu \rangle + \langle c, \hat{\pi} \rangle + \varepsilon \log \left(\left\langle e^{(\alpha_i + \beta_j - c_{ij} - \varepsilon)/\varepsilon}, 1 \right\rangle \right).$$

Proof. Consider the minimization problem of (A.2)

$$\inf_{\sum_{i,j} \pi_{ij} = 1, \pi_{ij} \geq 0} \left\{ \sum_{i,j} \sigma_{ij} \pi_{ij} - \varepsilon \sum_{i,j} \pi_{ij} \log \pi_{ij} + \sum_i \alpha_i \mu_i + \sum_j \nu_j \beta_j \right\},$$

with $\sigma_{ij} = -\alpha_i - \beta_j + c_{ij}$. Then we get from equation (A.2) with the extra constraint $\sum_{i,j} \pi_{ij} = 1$ with associated Lagrangian multiplier λ :

$$\mathcal{L}(\pi, \alpha, \beta) = \sum_{i,j} [\pi_{ij} \sigma_{ij} + \varepsilon \pi_{ij} \log \pi_{ij}] + \lambda \left(\sum_{i,j} \pi_{ij} - 1 \right) + \langle \alpha, \mu \rangle + \langle \beta, \nu \rangle.$$

Differentiating this with respect to π_{ij} gives us

$$\frac{\partial \mathcal{L}(\pi, \alpha, \beta)}{\partial \pi_{ij}} = \sigma_{ij} + \varepsilon \log \pi_{ij} + \varepsilon - \lambda.$$

Setting this derivative to zero at to minimize with respect to π , we get

$$\log \pi_{ij} = \exp \left(\frac{-\sigma_{ij} - \varepsilon + \lambda}{\varepsilon} \right). \quad (\text{A.8})$$

Then, with the additional constraint $\sum_{i,j} \pi_{ij} = 1$, we get

$$\sum_{i,j} \exp \left(\frac{-\sigma_{ij} - \varepsilon + \lambda}{\varepsilon} \right) = 1,$$

so, since λ and ε are independent of i and j :

$$\begin{aligned} \exp \left(\frac{\lambda}{\varepsilon} \right) \sum_{i,j} \exp \left(\frac{-\sigma_{ij} - \varepsilon}{\varepsilon} \right) &= 1 \\ \Rightarrow \lambda &= -\varepsilon \log \sum_{i,j} \exp \left(\frac{-\sigma_{ij} - \varepsilon}{\varepsilon} \right). \end{aligned}$$

With this definition of λ and filling in equation (A.8), we can see

$$\begin{aligned} &\sum_{i,j} \sigma_{ij} \pi_{ij} + \varepsilon \sum_{i,j} \pi_{ij} \log \pi_{ij} \\ &= \sum_{i,j} \sigma_{ij} \pi_{ij} + \varepsilon \sum_{i,j} \pi_{ij} \left(\frac{-\sigma_{ij} - \varepsilon + \lambda}{\varepsilon} \right) \\ &= \sum_{i,j} \sigma_{ij} \pi_{ij} - \sum_{i,j} \sigma_{ij} \pi_{ij} - \varepsilon \sum_{i,j} \pi_{ij} + \lambda \sum_{i,j} \pi_{ij} \\ &= -\varepsilon + \lambda \\ &= -\varepsilon \log \sum_{i,j} \exp \left(\frac{-\sigma_{ij} - \varepsilon}{\varepsilon} \right) - \varepsilon. \end{aligned}$$

Thus, together with equation (A.2), we can see for the dual problem of the entropy regularized OT with the extra constraint $\sum_{i,j} \pi_{ij} = 1$:

$$\max_{\alpha \in \mathbb{R}^m, \beta \in \mathbb{R}^n} \langle \alpha, \mu \rangle + \langle \beta, \nu \rangle - \varepsilon \log \sum_{i,j} \exp \left(\frac{\alpha_i + \beta_j - c_{ij} - \varepsilon}{\varepsilon} \right) - \varepsilon.$$

For convenience, let us call

$$D_2(\alpha, \beta) := \langle \alpha, \mu \rangle + \langle \beta, \nu \rangle - \varepsilon \log \sum_{i,j} \exp \left(\frac{\alpha_i + \beta_j - c_{ij} - \varepsilon}{\varepsilon} \right) - \varepsilon, \quad (\text{A.9})$$

and D_1 as D as in equation (4.4):

$$D_1(\alpha, \beta) := \langle \alpha, \mu \rangle + \langle \beta, \nu \rangle - \varepsilon \sum_{i,j} \exp \left(\frac{\alpha_i + \beta_j - c_{ij} - \varepsilon}{\varepsilon} \right). \quad (\text{A.10})$$

Furthermore, let us call

$$\begin{aligned} A(\alpha, \beta) &:= \langle \alpha, \beta \rangle + \langle \beta, \nu \rangle, \\ Z(\alpha, \beta, c) &:= \sum_{i,j} \exp \left(\frac{\alpha_i + \beta_j - c_{ij} - \varepsilon}{\varepsilon} \right). \end{aligned}$$

Then, we can see

$$\begin{aligned} D_1 &= A - \varepsilon Z, \\ D_2 &= A - \varepsilon \log Z - \varepsilon, \end{aligned}$$

so

$$D_2 = D_1 + \varepsilon Z - \varepsilon \log Z - \varepsilon,$$

and call

$$h(Z) := \varepsilon Z - \varepsilon \log Z - \varepsilon.$$

Note that $D_1 = D_2$ if and only if $Z = 1$. We will show that at the optimal value (α^*, β^*) , there holds $Z = 1$. We can see

$$\frac{\partial D_1}{\partial \alpha_i}(\alpha^*, \beta^*) = \mu_i - \sum_j \exp\left(\frac{\alpha_i^* + \beta_j^* - c_{ij}^* - \varepsilon}{\varepsilon}\right) = 0,$$

thus

$$\mu_i = \sum_j \exp\left(\frac{\alpha_i^* + \beta_j^* - c_{ij}^* - \varepsilon}{\varepsilon}\right).$$

Then, taking the sum over i , we can see

$$1 = \sum_i \mu_i = \sum_{i,j} \exp\left(\frac{\alpha_i^* + \beta_j^* - c_{ij}^* - \varepsilon}{\varepsilon}\right) = Z(\alpha^*, \beta^*, c^*).$$

Thus, at the optimum value there indeed holds $Z = 1$. We can do the same calculations for the derivative with respect to β and we will get the same result. Furthermore, for D_2 we can see

$$\frac{\partial D_2}{\partial \alpha_i}(\alpha^*, \beta^*) = \mu_i - \frac{\sum_j \exp\left(\frac{\alpha_i^* + \beta_j^* - c_{ij}^* - \varepsilon}{\varepsilon}\right)}{Z(\alpha^*, \beta^*, c^*)},$$

so if we sum over i we get

$$\sum_i \mu_i - \frac{\sum_{i,j} \exp\left(\frac{\alpha_i^* + \beta_j^* - c_{ij}^* - \varepsilon}{\varepsilon}\right)}{Z(\alpha^*, \beta^*, c^*)} = 1 - \frac{Z(\alpha^*, \beta^*, c^*)}{Z(\alpha^*, \beta^*, c^*)} = 0.$$

Thus we get indeed a zero for D_2 . Therefore, we can see

$$\max_{\alpha \in \mathbb{R}^m, \beta \in \mathbb{R}^n} D_1(\alpha, \beta) = \max_{\alpha \in \mathbb{R}^m, \beta \in \mathbb{R}^n} D_2(\alpha, \beta).$$

Now, we need to verify that all the statements of Theorem 4.4 still hold:

(i) $F(\alpha, \beta, c)$ is jointly convex in (α, β, c) :

Let us define $\phi = (\alpha, \beta, c)$, $\psi = (\delta\alpha, \delta\beta, \delta c)$ and

$$f(v) := E(\phi + v\psi).$$

Now, define $J : \mathbb{R}^{m \times n \times (m \times n)} \rightarrow \mathbb{R}$ as $(J\phi)_{ij} = \alpha_i + \beta_j - c_{ij} - \varepsilon$. Then, we can see

$$F(\phi) = \langle (-\mu, -\nu, \hat{\pi}), \phi \rangle + \varepsilon \log \langle e^{J\phi/\varepsilon}, 1 \rangle.$$

Now, let $\psi \in \mathbb{R}^{m \times n \times (m \times n)}$ and define the function $f : I \rightarrow \mathbb{R}$, with $I \subset \mathbb{R}$ an interval in a neighbourhood of 0, as

$$f(\eta) := F(\phi + \eta\psi),$$

with $\eta \in \mathbb{R}$.

Let us calculate the derivatives of f . Note that

$$f(\eta) := F(\phi + \eta\psi) = \langle (-\mu, -\nu, \hat{\pi}), \phi + \eta\psi \rangle + \varepsilon \log \langle e^{J(\phi + \eta\psi)/\varepsilon}, 1 \rangle.$$

Now, define

$$z(\eta) := \langle e^{J(\phi + \eta\psi)/\varepsilon}, 1 \rangle = \sum_{i,j} e^{(J(\phi + \eta\psi))_{ij}/\varepsilon},$$

and

$$p_{ij}(\eta) = \frac{e^{(J(\phi + \eta\psi))_{ij}/\varepsilon}}{z(\eta)}.$$

Note that

$$\sum_{i,j} p_{ij}(\eta) = \sum_{i,j} \frac{e^{(J(\phi + \eta\psi))_{ij}/\varepsilon}}{z(\eta)} = \frac{z(\eta)}{z(\eta)} = 1.$$

Then, we can see

$$\frac{d}{d\eta} \varepsilon \log(z(\eta)) = \varepsilon \frac{z'(\eta)}{z(\eta)} = \frac{\varepsilon}{z(\eta)} \sum_{i,j} e^{(J(\phi + \eta\psi))_{ij}/\varepsilon} \frac{(J\psi)_{ij}}{\varepsilon} = \sum_{i,j} p_{ij}(\eta) (J\psi)_{ij}.$$

Thus we can see

$$f'(\eta) = \langle (-\mu, -\nu, \hat{\pi}), \psi \rangle + \sum_{i,j} p_{ij}(\eta) (J\psi)_{ij}.$$

Now, note that

$$\begin{aligned} \frac{d}{d\eta} p_{ij}(\eta) &= \frac{z(\eta) \frac{d}{d\eta} e^{(J(\phi + \eta\psi))_{ij}/\varepsilon} - e^{(J(\phi + \eta\psi))_{ij}/\varepsilon} z'(\eta)}{z(\eta)^2} \\ &= \frac{z(\eta) e^{(J(\phi + \eta\psi))_{ij}/\varepsilon} \frac{(J\psi)_{ij}}{\varepsilon} - e^{(J(\phi + \eta\psi))_{ij}/\varepsilon} \sum_{i,j} e^{(J(\phi + \eta\psi))_{ij}/\varepsilon} \frac{(J\psi)_{ij}}{\varepsilon}}{z(\eta)^2} \\ &= p_{ij}(\eta) \frac{(J\psi)_{ij}}{\varepsilon} - p_{ij}(\eta) \cdot \frac{\sum_{p,q} e^{(J(\phi + \eta\psi))_{pq}/\varepsilon} \frac{(J\psi)_{pq}}{\varepsilon}}{z(\eta)} \\ &= \frac{1}{\varepsilon} \left[p_{ij}(\eta) (J\psi)_{ij} - p_{ij}(\eta) \sum_{p,q} p_{pq}(\eta) (J\psi)_{pq} \right] \end{aligned}$$

Thus we can see

$$\begin{aligned} f''(\eta) &= \sum_{i,j} p'_{ij}(\eta) (J\psi)_{ij} \\ &= \frac{1}{\varepsilon} \left[\sum_{i,j} p_{ij}(\eta) (J\psi)_{ij}^2 - \sum_{i,j} p_{ij}(\eta) \sum_{p,q} p_{pq}(\eta) (J\psi)_{pq} (J\psi)_{ij} \right] \\ &= \frac{1}{\varepsilon} \left[\sum_{i,j} p_{ij}(\eta) (J\psi)_{ij}^2 - \left(\sum_{i,j} p_{ij}(\eta) (J\psi)_{ij} \right)^2 \right] \end{aligned}$$

Note that

$$\begin{aligned} \left(\sum_{i,j} p_{ij}(\eta) (J\psi)_{ij} \right)^2 &= \left(\sum_{i,j} \sqrt{p_{ij}(\eta)} (J\psi)_{ij} \cdot \sqrt{p_{ij}(\eta)} \right)^2 \\ &\leq \sum_{i,j} p_{ij}(\eta) (J\psi)_{ij}^2 \cdot \sum_{i,j} p_{ij}(\eta) = \sum_{i,j} p_{ij}(\eta) (J\psi)_{ij}^2 \end{aligned}$$

through Cauchy-Schwartz and since $\sum_{i,j} p_{ij}(\eta) = 1$, per definition. So we can see $f''(\eta) \geq 0$ for any $\eta \in I$. Since ψ is arbitrary, we can see F is a convex functional of ϕ .

(ii) F is constant on each equivalence class $[(\alpha, \beta, c)]$:

Suppose $(\alpha, \beta, c) \sim (\bar{\alpha}, \bar{\beta}, \bar{c})$, and denote $\delta_{\alpha_i} := \alpha_i - \bar{\alpha}_i$ and $\delta_{\beta_j} := \beta_j - \bar{\beta}_j$. Then, we can see

$$\bar{c}_{ij} = \bar{\alpha}_i + \bar{\beta}_j - \alpha_i - \beta_j + c_{ij} = -\delta_{\alpha_i} - \delta_{\beta_j} + c_{ij}.$$

Hence, we have

$$\begin{aligned} F(\bar{\alpha}, \bar{\beta}, \bar{c}) &= \sum_{i,j} \bar{c}_{ij} \hat{\pi}_{ij} - \sum_i \bar{\alpha}_i \mu_i - \sum_j \bar{\beta}_j \nu_j + \varepsilon \log \sum_{i,j} e^{(\bar{\alpha}_i + \bar{\beta}_j - \bar{c}_{ij} - \varepsilon)/\varepsilon} \\ &= \sum_{i,j} (-\delta_{\alpha_i} - \delta_{\beta_j} + c_{ij}) \hat{\pi}_{ij} - \sum_i (\alpha_i - \delta_{\alpha_i}) \mu_i - \sum_j (\beta_j - \delta_{\beta_j}) \nu_j \\ &\quad + \varepsilon \log \sum_{i,j} e^{(\alpha_i + \beta_j - c_{ij} - \varepsilon)/\varepsilon} \\ &= F(\alpha, \beta, c) - \sum_{i=1}^n \delta_{\alpha_i} \left(\sum_{j=1}^m \hat{\pi}_{ij} \right) - \sum_{j=1}^m \delta_{\beta_j} \left(\sum_{i=1}^n \hat{\pi}_{ij} \right) + \sum_i \delta_{\alpha_i} \mu_i + \sum_j \delta_{\beta_j} \nu_j \\ &= F(\alpha, \beta, c). \end{aligned}$$

if we use the same result of cancellations as in the proof of Theorem 4.4. Thus F is a constant on the equivalence class $[(\alpha, \beta, c)]$.

(iii) $\tilde{F} : \widetilde{\mathcal{W}} \rightarrow \mathbb{R}$ with $\tilde{F}[(\alpha, \beta, c)] = F(\alpha, \beta, c)$ is well-defined:

As an immediate result of (ii), we can set $\tilde{F} : \widetilde{\mathcal{W}} \rightarrow \mathbb{R}$ as $\tilde{F}([\phi]) := F(\phi)$ for every $[\phi] \in \widetilde{\mathcal{W}}$, then $\tilde{F}$ is well-defined.

(iv) The optimal transport plan π^* is given by $\pi^* = e^{(\alpha^* + \beta^* - c^* - \varepsilon)/\varepsilon}$:

It follows directly from the proof of Lemma 4.1 and Theorem 4.4 that (α^*, β^*) is the optimal dual variable of the forward OT with cost c^* , and the optimal transport plan is given by

$$\pi^* = e^{(\alpha^* + \beta^* - c^* - \varepsilon)/\varepsilon}.$$

(v) $[(\alpha^*, \beta^*, c^*)]$ is the unique minimizer of $\tilde{F}$:

Now, we will show $[(\alpha^*, \beta^*, c^*)]$ is the unique minimizer of $\tilde{F}$. Define for nonzero $[\psi] \in \widetilde{\mathcal{W}}$ the variation $\tilde{f} : I \rightarrow \mathbb{R}$, with $I \subseteq \mathbb{R}$ again an open neighbourhood of 0, as follows:

$$\tilde{f}(\eta) = \tilde{F}([\phi] + \eta[\psi]) = \tilde{F}([\phi + \eta\psi]) = F(\phi + \eta\psi) = f(\eta),$$

where we used $[\phi] + \eta[\psi] = [\phi + \eta\psi]$ for all $\phi, \psi \in \mathcal{W}$, and we took $\eta \in \mathbb{R}$. Pick $\psi = (\delta_\alpha, \delta_\beta, \delta_c)$ such that $(J\psi)_{ij} = \delta_{\alpha_i} + \delta_{\beta_j} - \delta_{c_{ij}}$. We can see, as in (i) :

$$\tilde{f}''(\eta) = \frac{1}{\varepsilon} \left[\sum_{i,j} p_{ij}(\eta) (J\psi)_{ij}^2 - \left(\sum_{i,j} p_{ij}(\eta) (J\psi)_{ij} \right)^2 \right] \geq 0.$$

Now, since $[\psi] \neq 0$, $(J\psi)$ is not constant, thus we can see $\tilde{f}''(\eta) > 0$. Thus $\tilde{f}$ is strictly convex, thus it can at most have one global minimizer, which is by construction $[\phi^*]$.

Thus, we can conclude that the minimization problem in equation (4.12) has the same set of solutions as the minimization problem in equation (4.8). $\square$

B. Differences Between Algorithms

As Algorithm 2 is different than Algorithm 1 from Ma et al. (2020), we investigated the differences of the resulting cost matrix. For the non-uniform expansion, in Figure B.1a, and non-uniform collapse, in Figure B.1b, the differences between the cost matrix as retrieved from Algorithm 2 and the algorithm from Ma et al. (2020) are plotted. These cases both use $\text{exp} = 2$, $n = 4$ and $M_c = 10$. Note that the original algorithm of Ma et al. (2020) differs with a constant scaling of our algorithm, except for the M_c terms, which are a result of our choice of R . Furthermore, note that the figures are not scaled as in Section 5.1, as that would result in a uniform heat map.

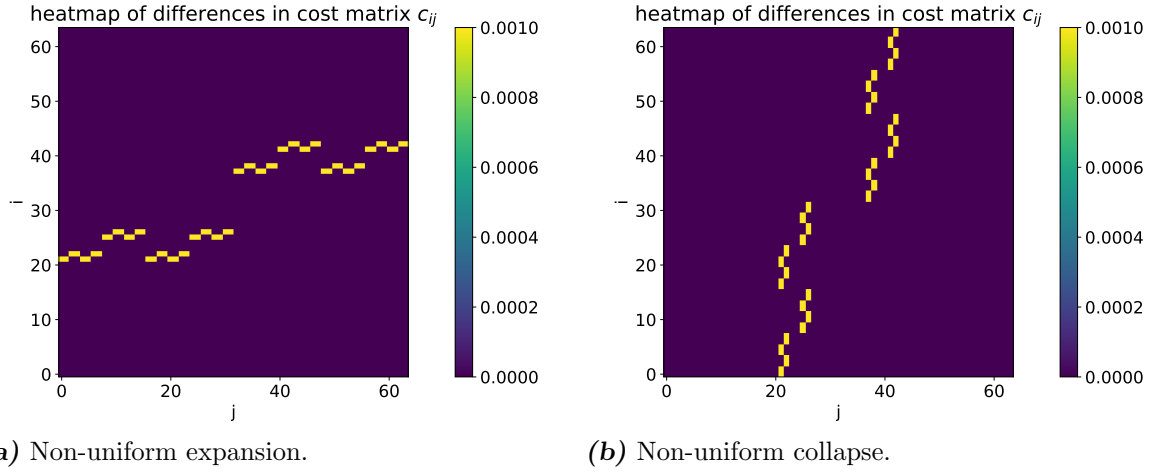


Figure B.1: Heat map of the differences in the cost matrix for non-uniform expansion (left) and collapse (right) from Algorithm 2 and the algorithm from Ma et al. (2020), for $\text{exp} = 2$ and $n = 4$.

C. Additional Reference Cases

In this appendix, we will discuss some more reference cases. Algorithm 2 is used to find the cost matrix of the simulations, with $R(c) = \mathbb{1}_U$, and $U = [0, M_c]^3$, as in Chapter 5. We choose $\gamma = \varepsilon = 1$ and $M_c = 10$. To better see differences in the heat map of the acquired cost matrix, we manually set all entries that were equal to M_c , equal to the highest entry of c that is not equal to M_c . All reference cases are simulated with one million particles. In all simulations, a box size of 100 (arbitrary units) is used.

We will look at the $n = 8$ case for the references for which it has not been discussed, as well as some values of `exp` for $n = 4$ which have not been discussed.

C.1 Voids

First, we look at voids, the non-uniform expansion. All data points start in a cube centered at $(50, 50, 50)$ with sides of 50, and particles can move towards the whole box. We simulate the same non-uniformity as in Section 5.1.2. We look at `exp` = 2 and `exp` = 5 for $n = 8$, in Figure C.1. We can see some structure, but notably the `exp` = 2 and `exp` = 5 cases look structure-wise very similar; the only difference is a (not necessarily uniform) scaling.

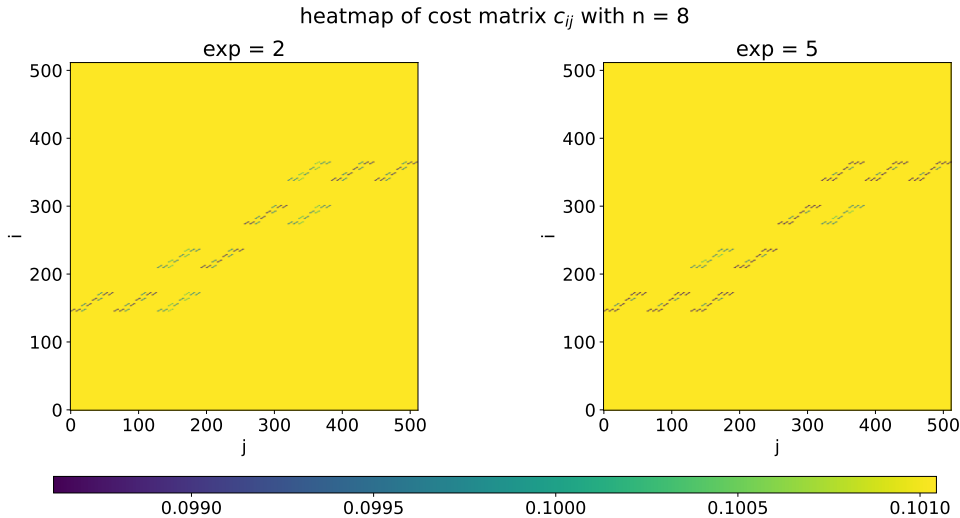


Figure C.1: Heat map of the cost matrix for non-uniform expansion, for both `exp` = 2 (left) and `exp` = 5 (right), for $n = 8$, as retrieved from Algorithm 2.

We can also look at other values of `exp`. For example, `exp` = 0.4 for $n = 4$ and $n = 8$, in Figure C.2. We can see that the $n = 4$ looks just like another scaling of what we already

had in Figure 5.4, and the structure in the $n = 8$ case is different, but not too different then in the $\text{exp} = 2$ or $\text{exp} = 5$ case in Figure 5.4.

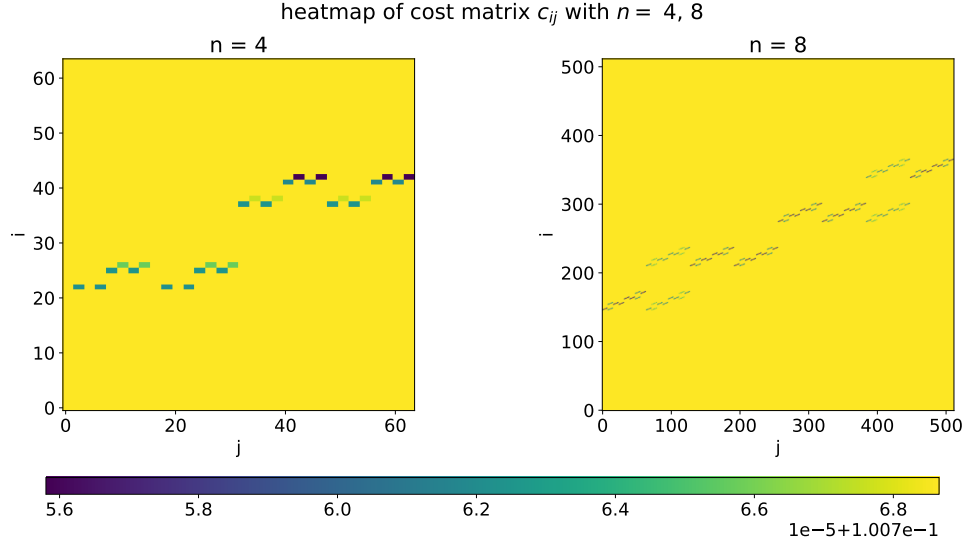


Figure C.2: Heat map of the cost matrix for non-uniform expansion, for $\text{exp} = 0.4$ for both $n = 4$ (left) $n = 8$ (right), as retrieved from Algorithm 2.

C.2 Clusters

Let us now look at clusters, specifically at the non-uniform collapse. All data points start in the whole cube and will move towards $(50, 50, 50)$, as in Section 5.1.3. We look at $\text{exp} = 2$ and $\text{exp} = 5$ for $n = 8$, in Figure C.3.

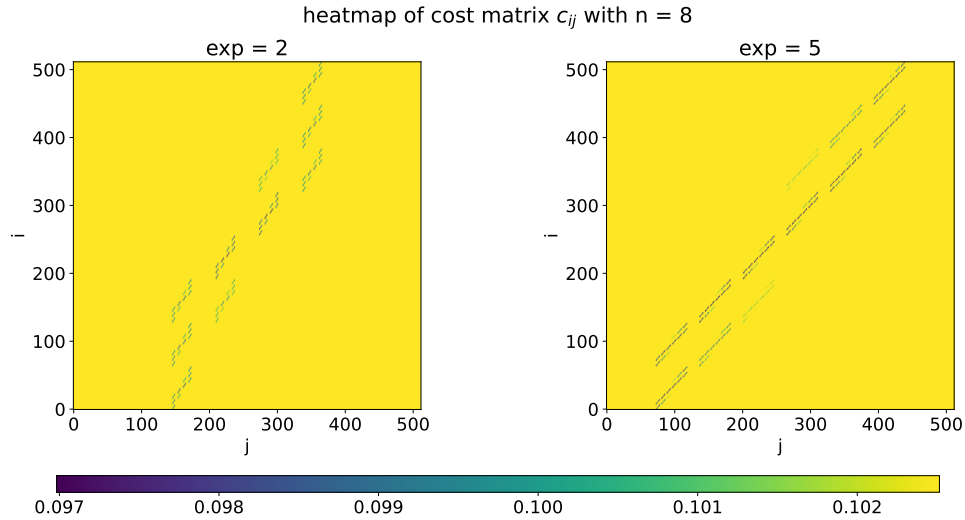


Figure C.3: Heat map of the cost matrix for non-uniform collapse, for both $\text{exp} = 2$ (left) and $\text{exp} = 5$ (right), for $n = 8$, as retrieved from Algorithm 2.

In the $n = 8$ case for non-uniform collapse, we can see some structure. Note, however, that the structure is significantly different in the $\text{exp} = 2$ case then in the $\text{exp} = 5$ case. This could have been expected from the $n = 4$ case in Figure 5.6, where for a higher value of exp a more tilted structure can be seen.

We can also look at other values of exp . For example, $\text{exp} = 0.4$ in Figure C.4 for $n = 4$ and $n = 8$. Note that the $\text{exp} = 0.4$ case looks a lot different than any other case, especially for the higher exp , for both $n = 4$ as for $n = 8$. This is probably due to the fact that the particle distribution changes as exp grows.

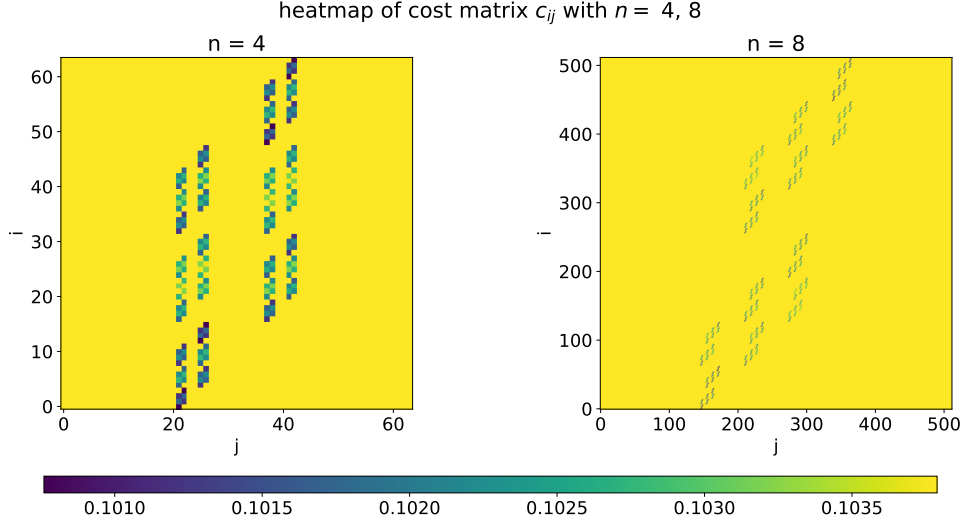


Figure C.4: Heat map of the cost matrix for non-uniform collapse, for $\text{exp} = 0.4$, for both $n = 4$ (left) and $n = 8$ (right), as retrieved from Algorithm 2.

C.3 Walls and Filaments

More cases of combinations of expansion and collapse, walls and filaments, were simulated too. All particles start in a cube of length 50 and can move towards the whole box (of length 100), as in Section 5.1. For walls, the cost matrix was calculated for $n = 8$, with $\text{exp} = 2$ and $\text{exp} = 5$, as can be seen in Figure C.5. Notably the $\text{exp} = 2$ and the $\text{exp} = 5$ do not seem to differ a lot apart from a scaling.

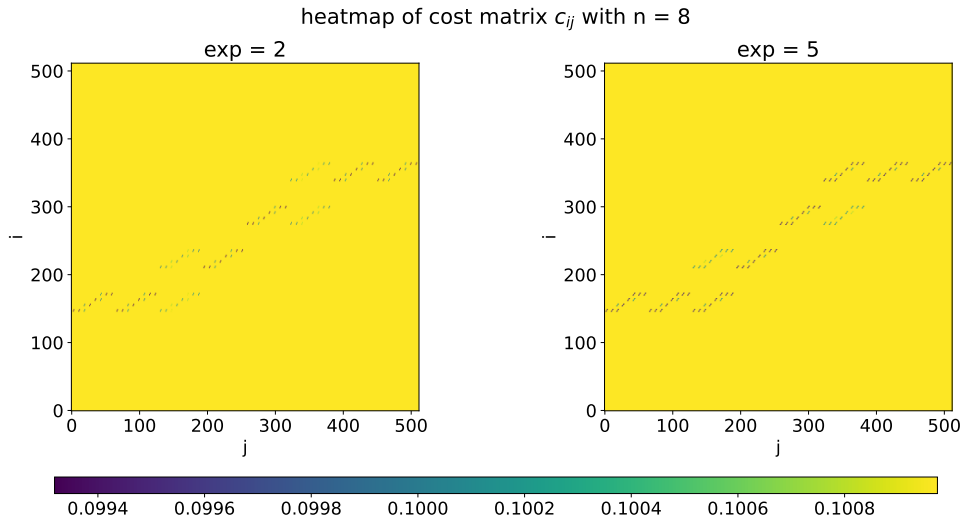


Figure C.5: Heat map of the cost matrix for non-uniform collapse, for both $\text{exp} = 2$ (left) and $\text{exp} = 5$ (right), for $n = 8$, as retrieved from Algorithm 2.

For filaments, the cost matrix was also calculated for $n = 8$, with $\text{exp} = 2$ and $\text{exp} = 5$, as can be seen in Figure C.6. In contrast to the walls, the $\text{exp} = 2$ case does look different than the $\text{exp} = 5$ case, by a rotation and a lack of some non- M_c entries.

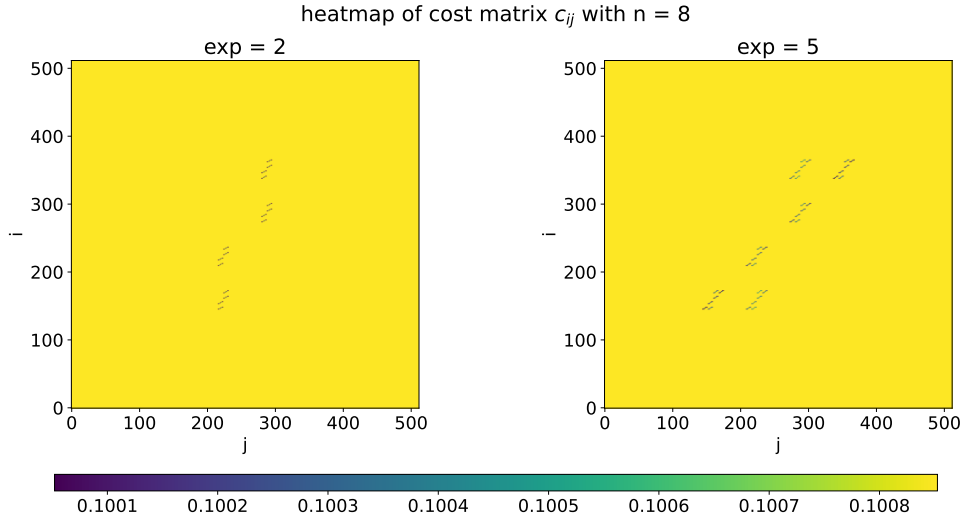


Figure C.6: Heat map of the cost matrix for non-uniform collapse, for both $\text{exp} = 2$ (left) and $\text{exp} = 5$ (right), for $n = 8$, as retrieved from Algorithm 2.